

L4: Statistical time series analysis for market classification

Course Code: COMP7415

Agenda

- Common classification for financial market
- Market prediction with statistical models
 - Simple Linear Regression
 - Multiple Linear Regression
 - ARIMA
 - Hurst Exponent
- Martingale Strategy

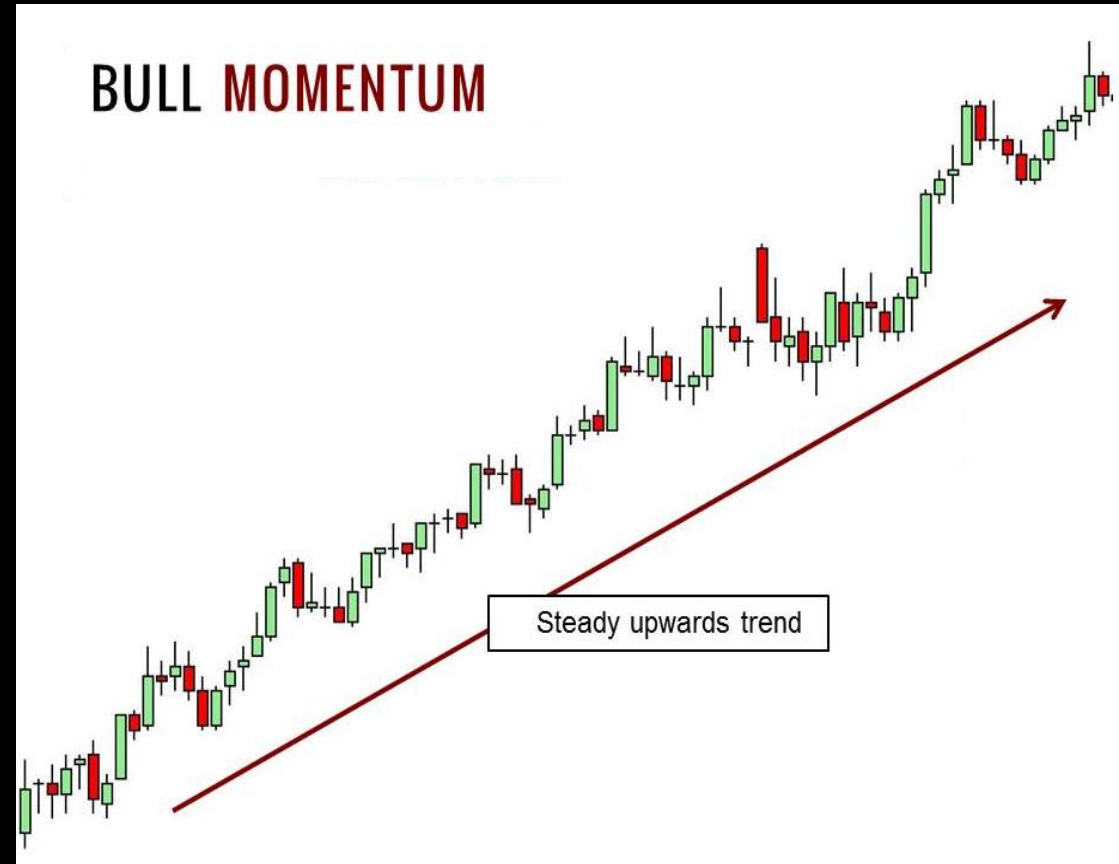
Market Classification

The financial market can generally be classified into

1. Trending (Momentum)
2. Mean Reversion
3. Random Walk

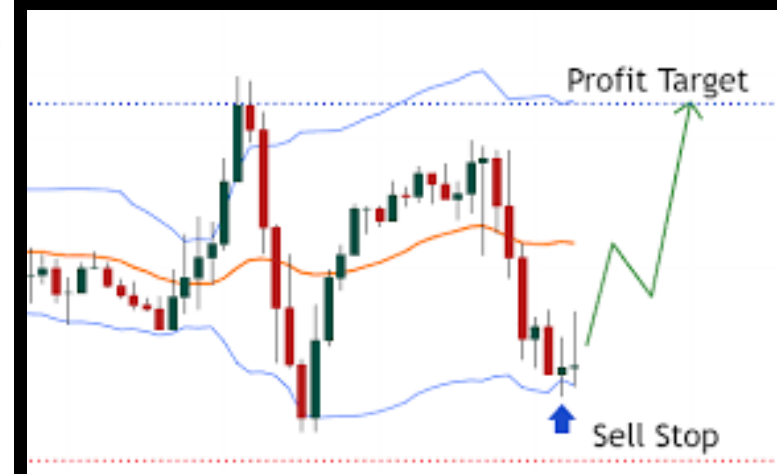
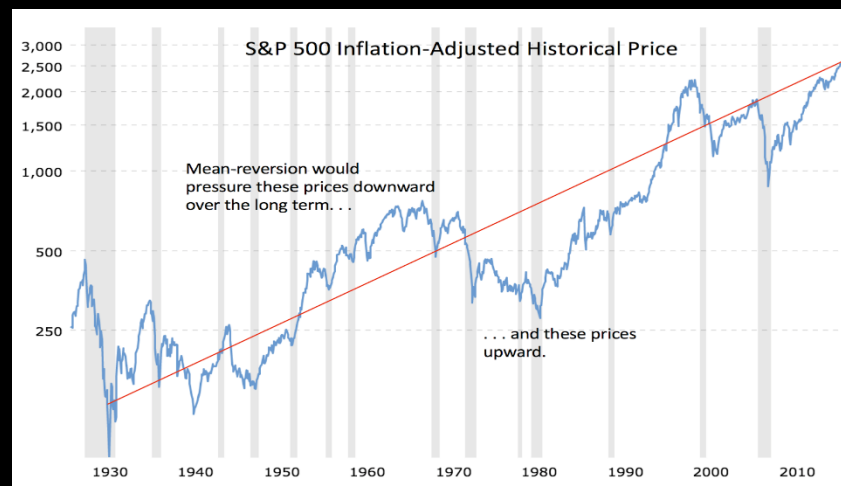
Trending (Momentum)

- Momentum trading involves
 - buying securities that are trending upwards and selling them when they appear to have peaked, or
 - shorting securities that are trending downwards and covering them when they appear to have bottomed out.

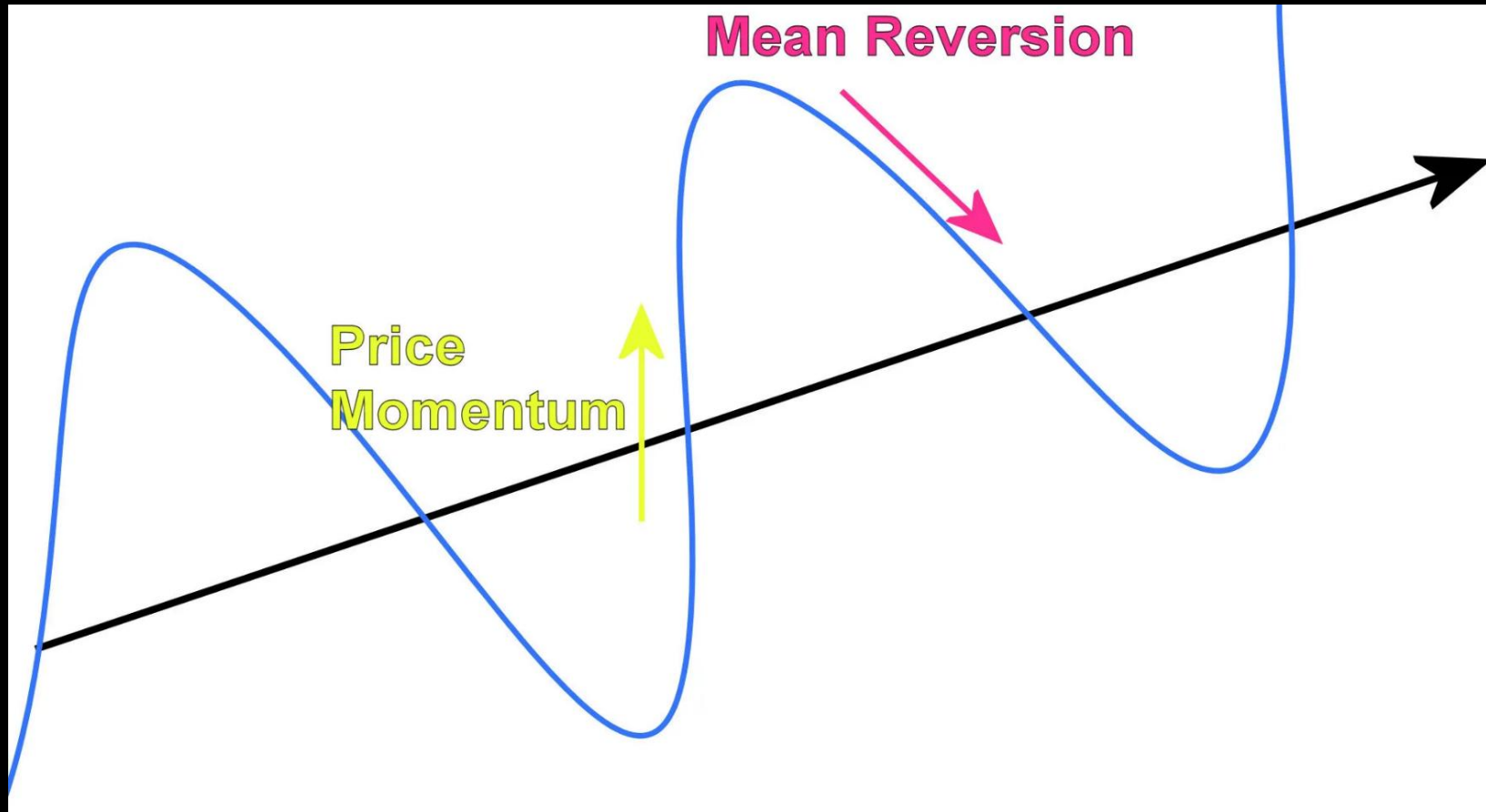


Mean Reversion

- Mean reversion suggests that asset prices and historical returns eventually revert to their long-term mean or average level.
- Based on the idea that extreme price movements are temporary and will revert to the mean.



Trending and Mean Reversion can occur at the same time



Random Walk

- It implies that price changes are random and do not follow any patterns or trends.
 - **Efficient Market Hypothesis (EMH)**: Underpins the idea that all known information is already reflected in stock prices, making it impossible to consistently achieve higher returns than average market returns.
 - **Independence**: Each price change is independent of previous price changes.
 - **Unpredictability**: Future price movements cannot be predicted based on past price movements.
- If the market is random, can you think of a trading strategy that can be profitable?

Time series Analysis

Introduction to Time Series Analysis

- Time series analysis involves examining data points collected or recorded at specific time intervals.
- It aims to identify patterns, trends, and other characteristics in the data.
- Examples:
 - Stock prices over time
 - Monthly unemployment rates
 - Daily temperature readings
 - Quarterly GDP growth rates

Simple Linear Regression

For market trend identification

Simple Linear Regression Model

- Linear regression is a statistical method for modeling the **linear** relationship between a dependent variable and independent variable.
- Key Concepts:
 - **Dependent Variable (Y)**: The variable we are trying to predict or explain.
 - **Independent Variable (X)**: The variable we use to make predictions.
 - **Linear Equation**:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

β_0 : Intercept

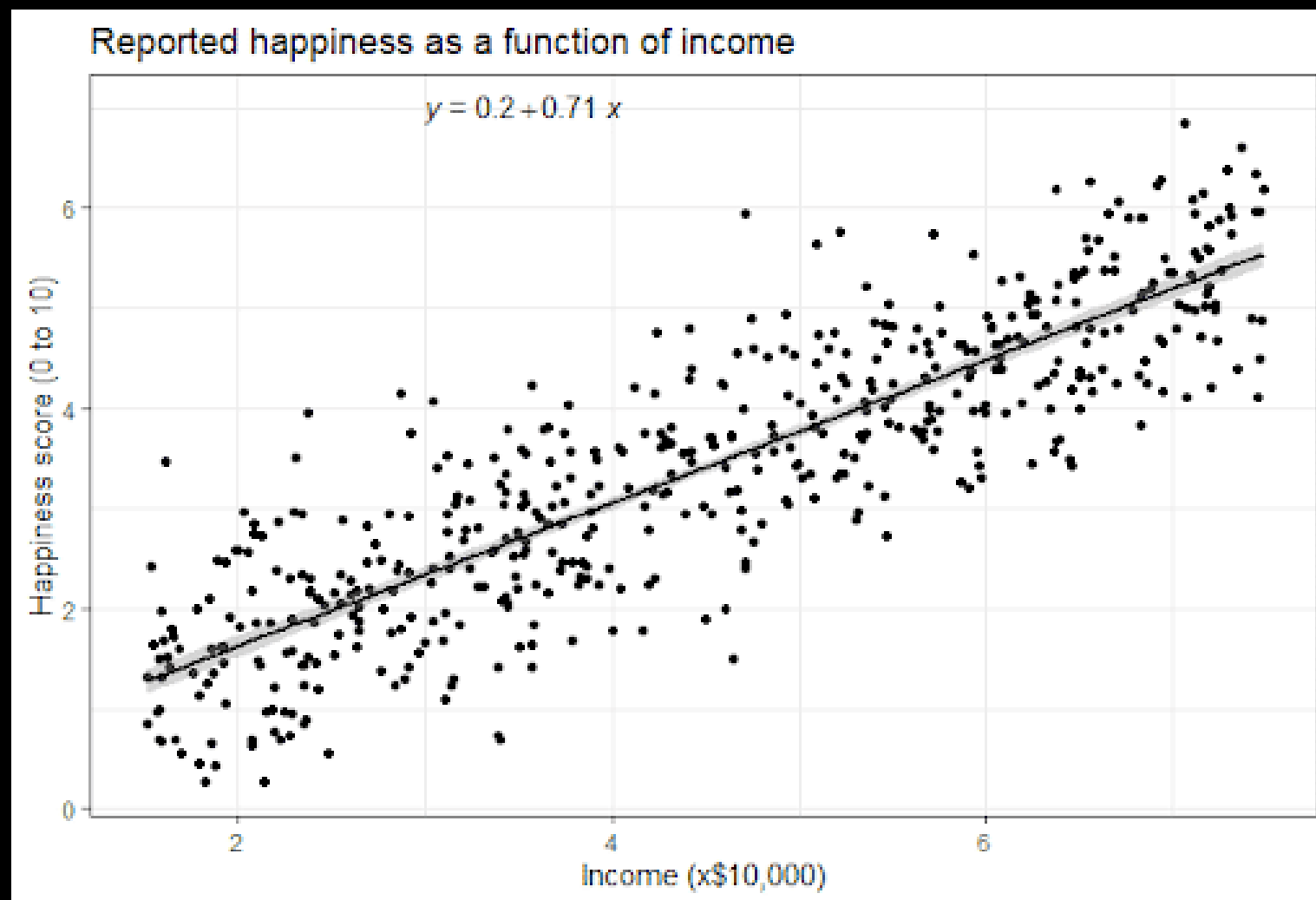
β_1 : Slope

ϵ : Error term

Why Use Linear Regression?

- **Simplicity:** Easy to understand and implement.
- **Interpretability:** Coefficients provide insights into the relationship between variables.
- **Efficiency:** Computationally efficient for small to medium-sized datasets.
- **Foundation:** Basis for more complex models.

Example



Parameter Estimation

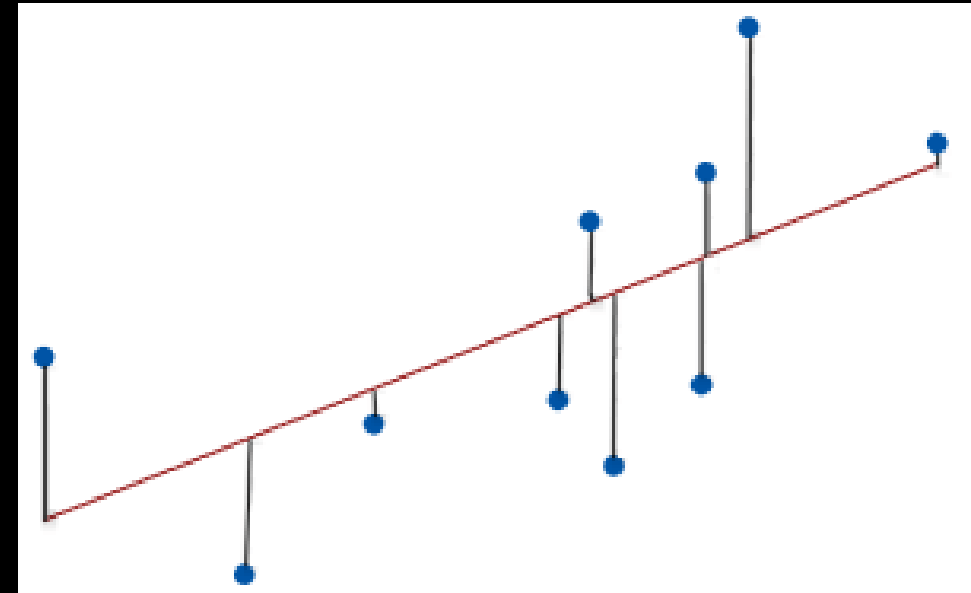
- Define
 - y_i is the actual value
 - $\hat{y}_i$ is the predicted value
 - N is the Number of observation
 - $\bar{X}, \bar{Y}$ are the mean of $\{x_i\}$ and $\{y_i\}$ respectively

- Residual is the difference between actual and model estimated value

$$e_i = y_i - \hat{y}_i$$

- Mean squared error represents the average squared residual

$$MSE = \frac{\sum e_i^2}{n - 2} = \frac{\sum (y_i - \hat{y}_i)^2}{n - 2} \quad \leftarrow \text{degree of freedom}$$



Parameter Estimation

- Predicted value: $\hat{y}_i = \beta_0 + \beta_1 x_i$
- Sum of Squared Error (SSE):

$$SSE = \sum (y_i - \hat{y}_i)^2 = \sum (y_i - (\beta_0 + \beta_1 x_i))^2$$

- Take partial derivatives, and set to zero to solve for β_0 and β_1

$$\frac{\partial SSE}{\partial \beta_0} = -2 \sum (y_i - \beta_0 - \beta_1 x_i) = 0$$

$$\frac{\partial SSE}{\partial \beta_1} = -2 \sum (y_i - \beta_0 - \beta_1 x_i) x_i = 0$$



$$\beta_0 = \bar{Y} - \beta_1 \bar{X}$$

$$\beta_1 = \frac{\sum (x_i - \bar{X})(y_i - \bar{Y})}{\sum (x_i - \bar{X})^2}$$

Distribution of β_0, β_1

- Assume error term follows identical independent normal distribution (i.i.d.)

$$\epsilon \sim N(0, \sigma^2)$$

- Unbiased estimate of σ^2 is

$$s^2 = MSE = \frac{SSE}{n-2} = \frac{\sum (y_i - \hat{y}_i)^2}{n-2}$$

- Standard error of β_1

$$SE(\beta_1) = \sqrt{\frac{s^2}{\sum (x_i - \bar{X})^2}}$$

- Standard error of β_0

$$SE(\beta_0) = s \sqrt{\frac{1}{n} + \frac{\bar{X}^2}{\sum (x_i - \bar{X})^2}}$$

Confidence Interval

- A $100(1-\alpha)\%$ confidence interval for β_1

$$\widehat{\beta}_1 \pm t_{\alpha/2, n-2} \sqrt{\frac{s^2}{\sum (x_i - \bar{X})^2}}$$

- A $100(1-\alpha)\%$ confidence interval for β_0

$$\widehat{\beta}_0 \pm t_{\alpha/2, n-2} \times s \sqrt{\frac{1}{n} + \frac{\bar{X}^2}{\sum (x_i - \bar{X})^2}}$$

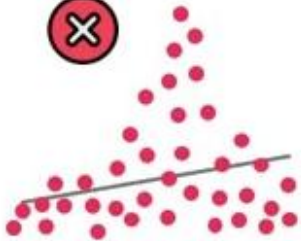
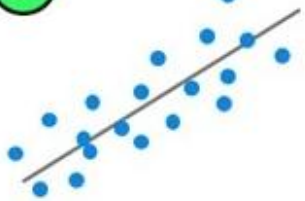
Model Assumptions

1. **Linear Relationship:** the existence of a linear relationship between the dependent variable and the independent variables. This linearity can be visually inspected using scatterplots, which should reveal a straight-line relationship rather than a curvilinear one.
2. **Multivariate Normality:** it assumes that the residuals are normally distributed. This assumption can be assessed by examining histograms or Q-Q plots of the residuals, or through statistical tests such as the Kolmogorov-Smirnov test.
3. **Independence:** each observation is independent of the others
4. **No Multicollinearity:** the independent variables $\{X_1, X_2, \dots\}$ are not highly correlated with each other. This can be checked using correlation matrices.
5. **Homoscedasticity:** The variance of error terms (residuals) should be consistent across all levels of the independent variables. A scatterplot of residuals versus predicted values should not display any discernible pattern, such as a cone-shaped distribution, which would indicate heteroscedasticity

Model Assumptions

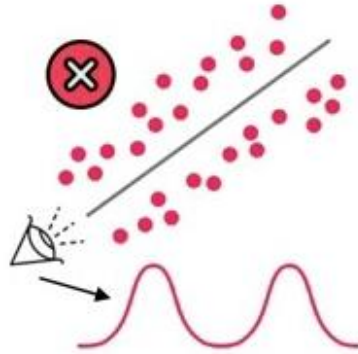
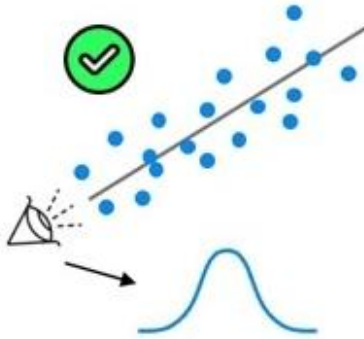
1. Linearity

(Linear relationship between Y and each X)



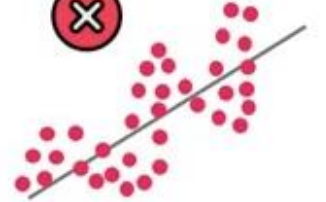
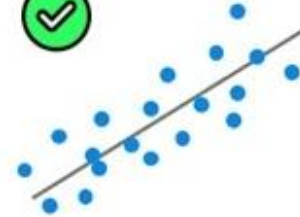
2. Multivariate Normality

(Normality of error distribution)



3. Independence

(of observations. Includes "no autocorrelation")



4. Lack of Multicollinearity

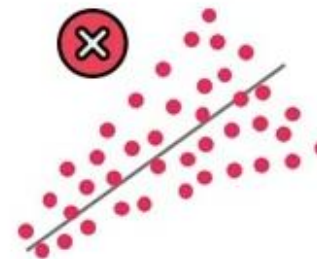
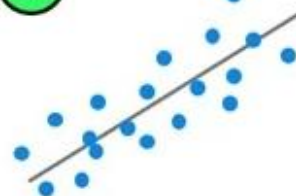
(Predictors are not correlated with each other)

$$X_1 \not\sim X_2$$

$$X_1 \sim X_2$$

5. Homoscedasticity

(Equal variance)



Model Effectiveness

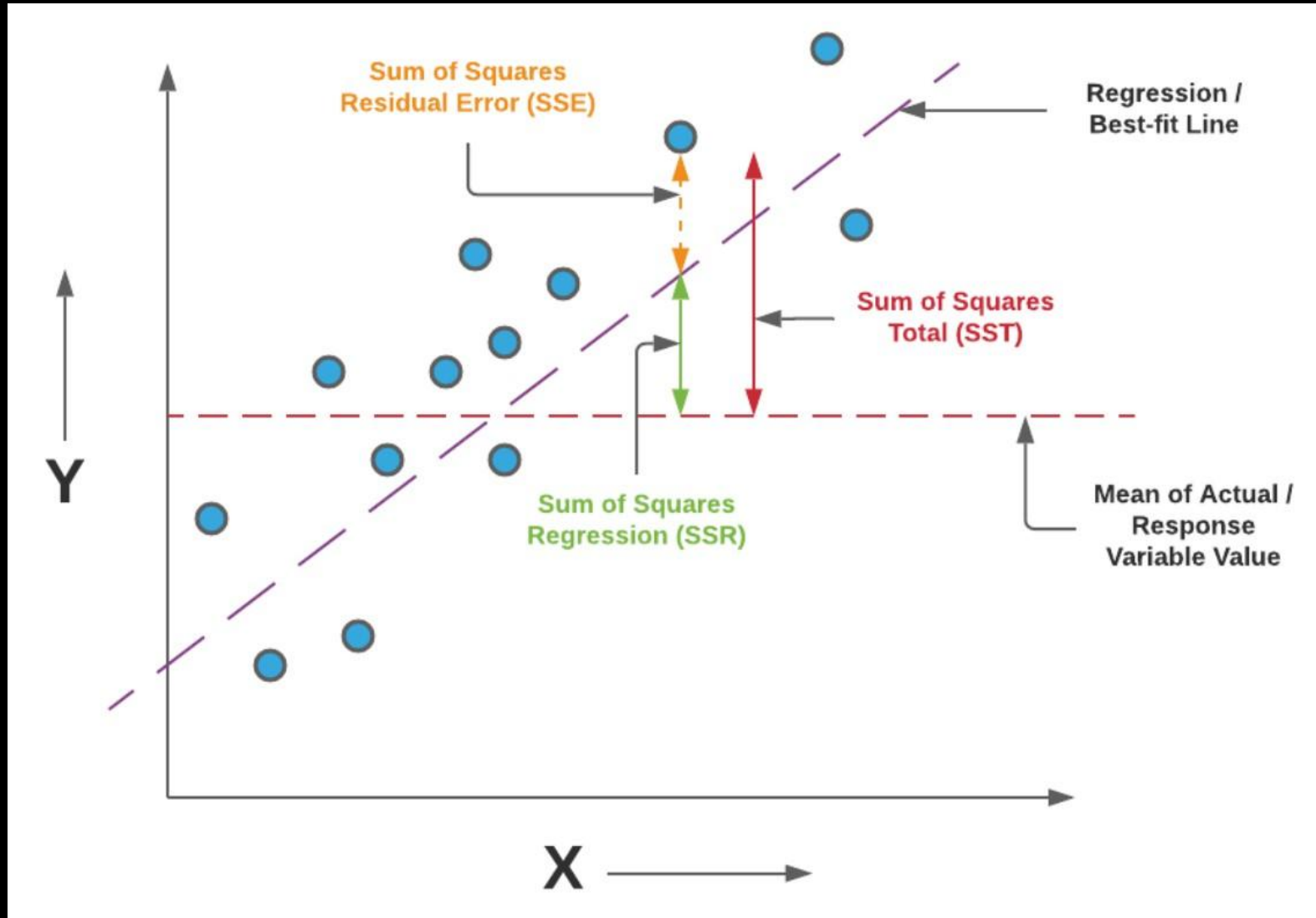
- R-Squared (Coefficient of Determination)
 - represents the proportion of the variance in the dependent variable that is predictable from the independent variable(s).
 - Indicates how well the data points fit a regression line.
- Interpretation
 - R^2 ranges from 0 to 1.
 - $R^2 = 1$: Perfect fit (the model explains all the variability of the response data around its mean).
 - $R^2 = 0$: The model does not explain any of the variability of the response data around its mean.
- Formula

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

$$SS_{res} \text{ (Sum of squared residuals)} = \sum (y_i - \hat{y}_i)^2$$

$$SS_{tot} \text{ (Total Sum of Squares)} = \sum (y_i - \bar{Y})^2$$

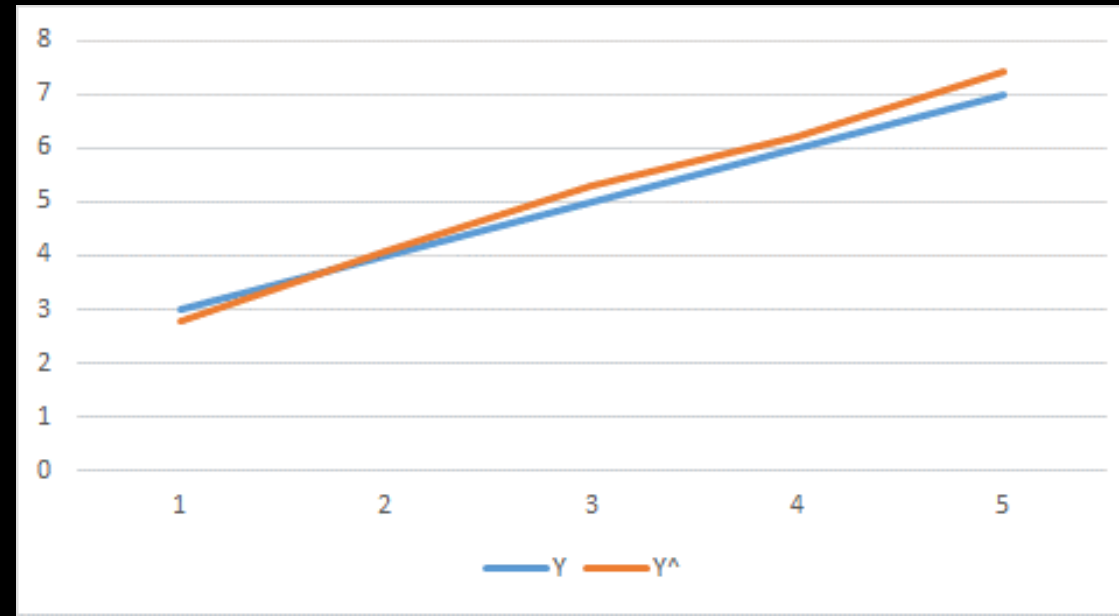
Breakdown of Regression Errors



R^2 Example

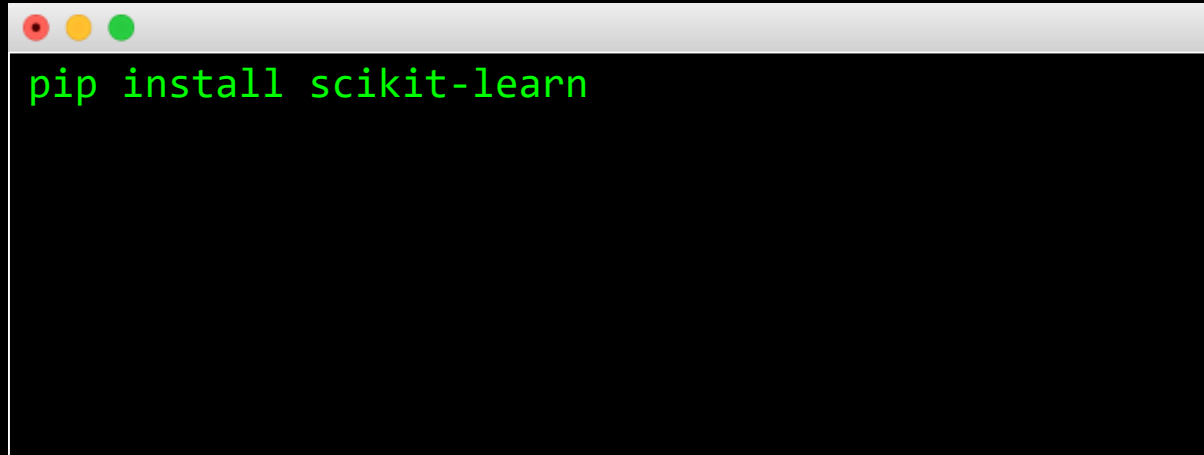
- Suppose we have the following data points:
 - Actual values (Y): [3, 4, 5, 6, 7]
 - Predicted values ($\hat{Y}$): [2.8, 4.1, 5.3, 6.2, 7.4]
- $SS_{res} = (3-2.8)^2 + (4-4.1)^2 + (5-5.3)^2 + (6-6.2)^2 + (7-7.4)^2 = 0.37$
- $SS_{tot} = (3-5)^2 + (4-5)^2 + (5-5)^2 + (6-5)^2 + (7-5)^2 = 10$

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{0.37}{10} = 0.963$$



Scikit-learn

- Official website (<https://scikit-learn.org/stable/api/index.html>)

A terminal window with a light gray title bar containing three colored window control buttons (red, yellow, green). The terminal has a black background and displays the command `pip install scikit-learn` in green text.

```
pip install scikit-learn
```

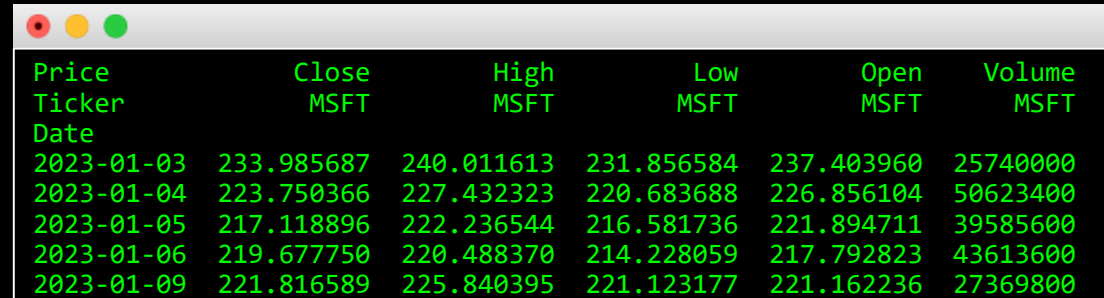
Python Example (1): download MSFT data

```
import pandas as pd
import yfinance as yf

# Fetch historical data for Microsoft (MSFT)
msft_data = yf.download('MSFT', start='2023-01-01', end='2023-12-31')

# Display the first few rows of the dataset
print(msft_data.head())

# Preprocess the data
msft_data['Date'] = msft_data.index
msft_data['Date'] = pd.to_datetime(msft_data['Date'])
msft_data.set_index('Date', inplace=True)
```

A terminal window with a light gray title bar and three colored window control buttons (red, yellow, green) on the left. The terminal displays a table of Microsoft (MSFT) stock data for the first five trading days of 2023. The data is presented in a clean, monospaced font with green text on a black background. The table has six columns: Price, Close, High, Low, Open, and Volume, each followed by a sub-header 'MSFT'. The rows show dates from 2023-01-03 to 2023-01-09.

Price Ticker Date	Close MSFT	High MSFT	Low MSFT	Open MSFT	Volume MSFT
2023-01-03	233.985687	240.011613	231.856584	237.403960	25740000
2023-01-04	223.750366	227.432323	220.683688	226.856104	50623400
2023-01-05	217.118896	222.236544	216.581736	221.894711	39585600
2023-01-06	219.677750	220.488370	214.228059	217.792823	43613600
2023-01-09	221.816589	225.840395	221.123177	221.162236	27369800

Python Example (2): fit LR model

```
from sklearn.linear_model import LinearRegression

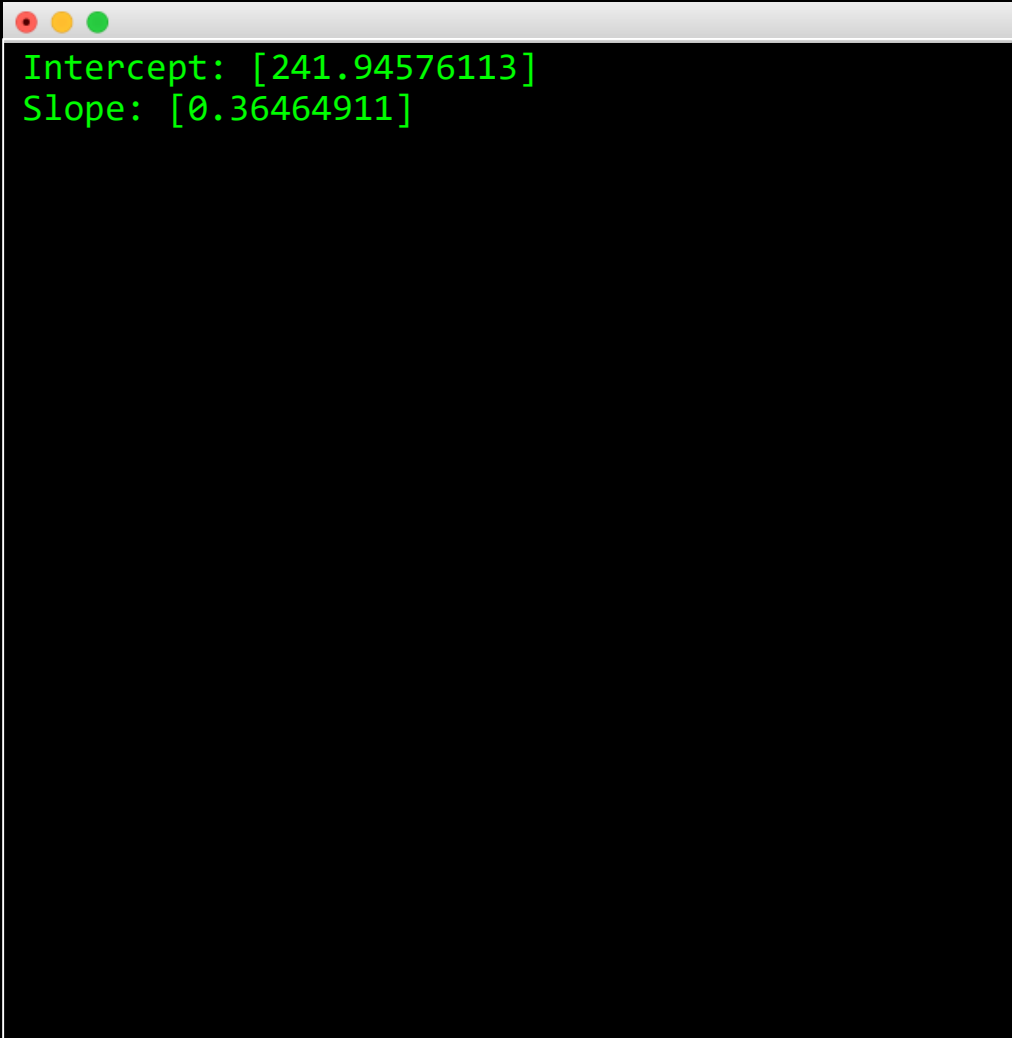
# Prepare the features and target variable
msft_data['Day'] = msft_data.index.dayofyear
X = msft_data[['Day']]
y = msft_data['Close']

# Split the data into training and testing sets
n_train = int(len(X)*0.8)
n_test = len(X) - n_train

X_train, X_test = X[0:n_train], X[-n_test:]
y_train, y_test = y[0:n_train], y[-n_test:]

# Initialize and train the linear regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Display the coefficients
print(f"Intercept: {model.intercept_}")
print(f"Slope: {model.coef_[0]}")
```

A terminal window with a light gray title bar and three colored window control buttons (red, yellow, green) on the left. The terminal has a black background with green text. It displays the output of the linear regression model fit: 'Intercept: [241.94576113]' and 'Slope: [0.36464911]'.

```
Intercept: [241.94576113]
Slope: [0.36464911]
```

Python Example (3): model evaluation

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
# Make predictions on the test set
```

```
y_pred0 = model.predict(X_train)
```

```
y_pred = model.predict(X_test)
```

```
# Calculate evaluation metrics
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print(f"Mean Squared Error: {mse}")
```

```
print(f"R-squared: {r2}")
```

```
# Plot the results
```

```
import matplotlib.pyplot as plt
```

```
plt.style.use('dark_background')
```

```
# Plot the results (training set)
```

```
plt.figure(figsize=(10, 6))
```

```
plt.scatter(X_train, y_train, color='blue', label='Actual Prices')
```

```
plt.plot(X_train.to_numpy(), y_pred0, color='red', linewidth=2, label='Predicted Prices')
```

```
plt.xlabel('Day of Year')
```

```
plt.ylabel('Closing Price (USD)')
```

```
plt.title('MSFT Stock Price Prediction (Training Dataset)')
```

```
plt.legend()
```

```
plt.show()
```

```
# Plot the results (test set)
```

```
plt.figure(figsize=(10, 6))
```

```
plt.scatter(X_test, y_test, color='blue', label='Actual Prices')
```

```
plt.plot(X_test.to_numpy(), y_pred, color='red', linewidth=2, label='Predicted Prices')
```

```
plt.xlabel('Day of Year')
```

```
plt.ylabel('Closing Price (USD)')
```

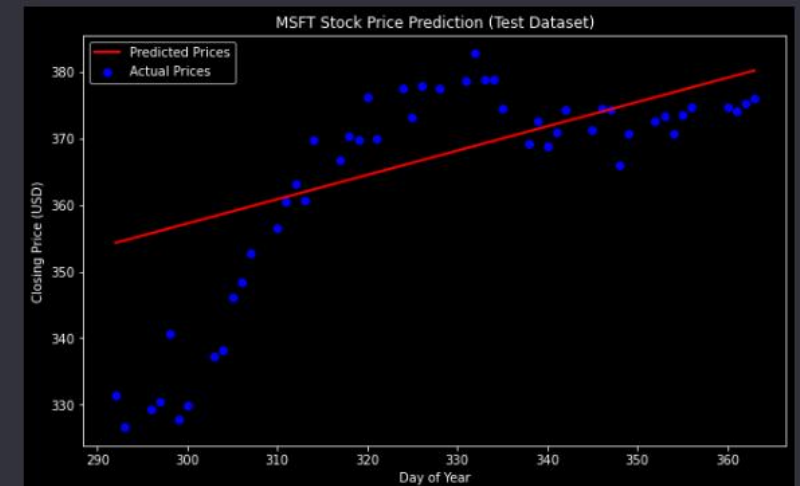
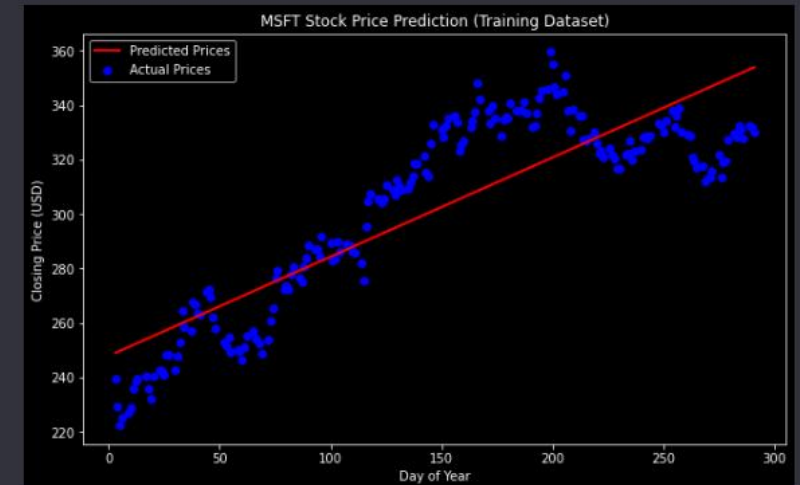
```
plt.title('MSFT Stock Price Prediction (Test Dataset)')
```

```
plt.legend()
```

```
plt.show()
```

Mean Squared Error: 144.25272079512234

R-squared: 0.46490145270565975



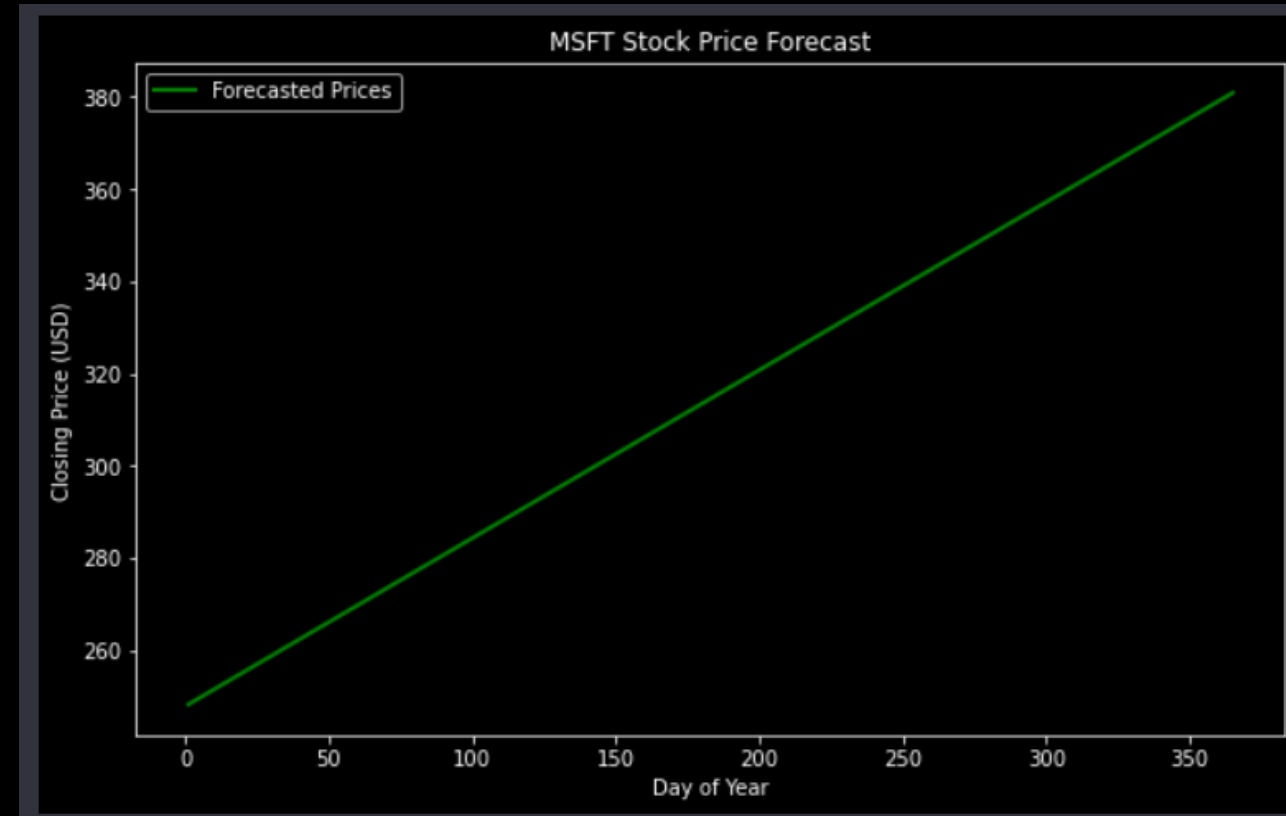
Python Example (4): future price forecast

```
import numpy as np

# Generate future days for forecasting
future_days = np.array(range(1, 366)).reshape(-1, 1)

# Predict future prices
future_prices = model.predict(future_days)

# Plot the forecast
plt.figure(figsize=(10, 6))
plt.plot(future_days, future_prices, color='green', linewidth=2, label='Forecasted Prices')
plt.xlabel('Day of Year')
plt.ylabel('Closing Price (USD)')
plt.title('MSFT Stock Price Forecast')
plt.legend()
plt.show()
```



Linear Regression Trading Strategy

- Collect the previous N observations and fit a linear regression model
- Strategy 1:
 - Take LR model's predicted values as the fair/reference market price.
 - If current market price is much lower than the fair price, open LONG position
 - If current market price is much higher than the fair price, open SHORT position
- Strategy 2:
 - Project the future prices using the LR model
 - If the future price is projected to be higher than the current price, open LONG position
 - If the future price is projected to be lower than the current price, open SHORT position

Backtest LR Strategy

- Instrument: MSFT
- Initial capital: 1000 USD
- Period: 2023.01.01 – 2023.12.31
- Data Interval: 1 day
- Short-selling allowed: No

```
from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

class AlgoEvent:
    def __init__(self):
        self.numObs = 30

    def start(self, mEvt):
        self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)
        self.evt.start()

    def on_marketdatafeed(self, md, ab):
        # get previous 100 days closing price
        obs = self.evt.getHistoricalBar(contract={"instrument": "MSFT"}, numOfBar=self.numObs, interval="D",
timestamp=md.timestamp)
        Y = [obs[t]['c'] for t in obs]
        X = np.array(range(0, self.numObs)).reshape(-1, 1)

        # fit ARIMA model
        model = LinearRegression()
        model.fit(X, Y)
        # evaluate model effectiveness
        Y_pred = model.predict(X)
        mse = mean_squared_error(Y, Y_pred)
        r2 = r2_score(Y, Y_pred)
        self.evt.consoleLog('mse, r-square = ', mse, r2)

        # entry if e-square > 0.5
        if r2>0.5:
            # 5 day forecast
            future_days = np.array(range(1, 5)).reshape(-1, 1)
            forecast = model.predict(future_days)
            future_price = forecast[-1]
            self.evt.consoleLog(forecast)

            # open buy order if future price is higher than current price
            if future_price>md.askPrice:
                order = AlgoAPIUtil.OrderObject(
                    instrument = md.instrument,
                    openclose = 'open',
                    buysell = 1, #buy
                    ordertype = 0, #market order
                    volume = 1, #1 lot
                    holdtime = 24*60*60*5 #holding period for 5 days
                )
                self.evt.sendOrder(order)
```

Backtest LR Strategy

Monthly Return:

Year	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	YTD
2023	4.52%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	-1.61%	-0.01%	0.97%	0.0%	0.0%	3.82%

Performance Statistics: ⓘ

No. of Tradable Days:	272	No. of Win Days:	20	No. of Loss Days:	14
Win Rate:	58.8235%	Max. Consecutive Win Day:	7	Max. Consecutive Loss Day:	4
Odd Ratio:	1.4286	Max. Consecutive Gains:	42.54	Max. Consecutive Loss:	-37.05
No. of Trades:	32	Average Consecutive Win Day:	1.93	Average Consecutive Loss Day:	0.1
Total PnL:	38.23	Average Per Trade PnL:	1.19	Average Per Day PnL:	0.14
Mean Daily Return:	0.0144%	Median Daily Return:	0.0%	Mean Annual Return:	3.6324%
Daily Return StdDev:	0.3043%	25th percentile Daily Return:	0.0%	75th percentile Daily Return:	0.0%
Daily Return Downside StdDev:	0.2275%	95% 1 day return VaR:	-0.0055%	99% 1 day return VaR:	-0.4191%
Daily Sharpe Ratio:	0.0474	Annual Sharpe Ratio:	0.7519	Max. Drawdown Amount:	37.05
Daily Sortino Ratio:	0.0634	Annual Sortino Ratio:	1.0057	Max. Drawdown Percent:	3.5215%
Max. Drawdown Duration:	97	Average Drawdown Duration:	17.73	Annual Volatility:	4.8115%
Gross Profit:	61.479	Gross Loss:	-23.931	Profit Factor:	2.569
Jensen Alpha:	0.0	Beta:	0.0	Information Ratio:	0.0
Omega Ratio:	0.0	Treynor Ratio:	0.0	Tail Ratio:	28.0459
Calmar Ratio:	1.0315	Average Holding Day:	5.5625	Annual Turnover Rate:	403.8137%

Multiple Linear Regression Model

MLR Model

- It is an extension of the simple linear regression model.
- **Dependent Variable (Y)**: The outcome we are trying to predict (eg. stock price).
- **Independent Variables ($X_1, X_2, \dots, X_p$)**: The predictors or features that influence Y (eg. economic indicators, technical indicators).

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \epsilon$$

β_0 : Intercept

$\beta_1, \beta_2, \dots, \beta_p$: coefficients

ϵ : Error term

MLR Model

- For p independent variables and n observations,
- In matrix presentation,

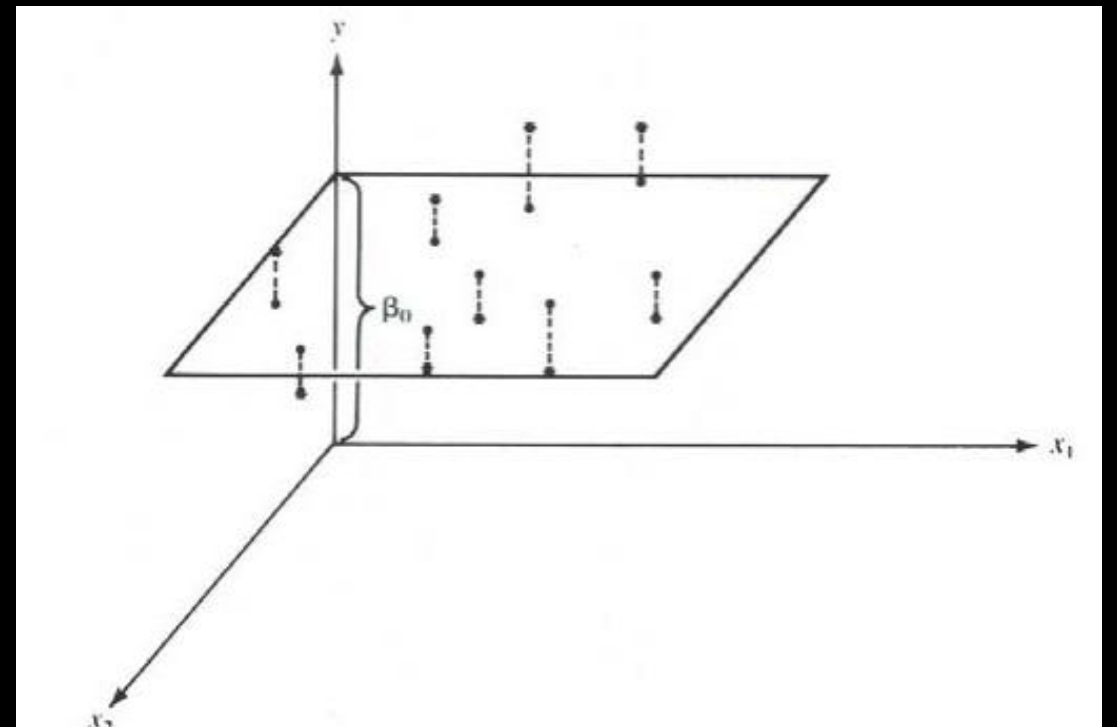
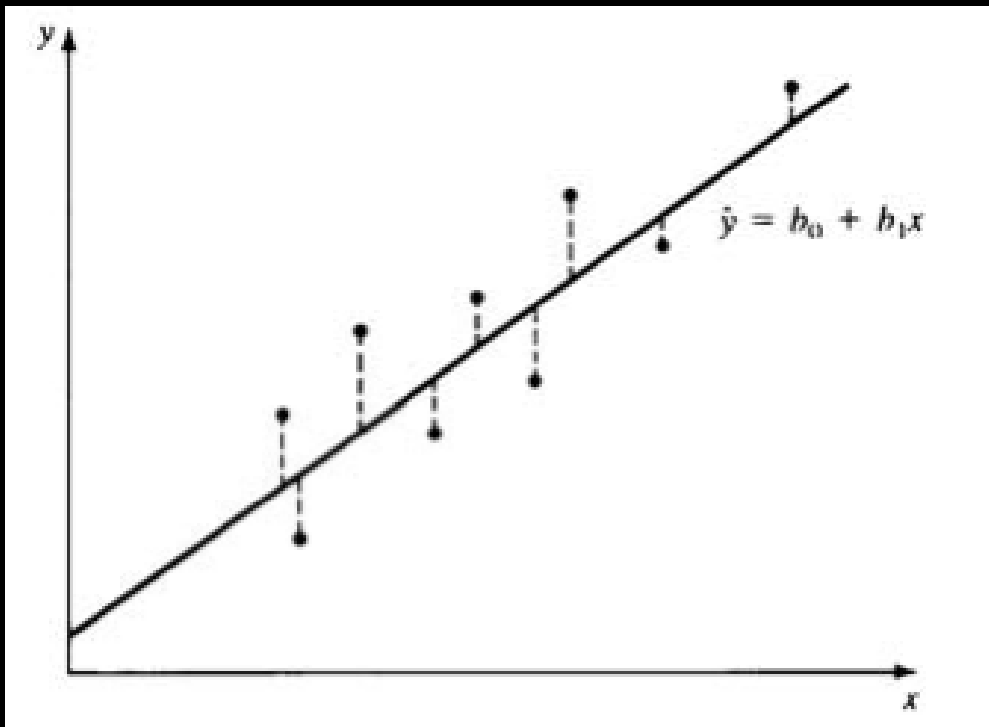
$$\underline{Y} = \underline{X}\underline{\beta} + \underline{\epsilon}$$

where

$$\underline{Y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \quad \underline{X} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1p} \\ \vdots & & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{pmatrix} \quad \underline{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix} \quad \underline{\epsilon} = \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

Regression Plane

- Find a linear plane such that the total distance between the data points and the plane is the smallest



Least Square Estimation of β

- Fitted value

$$\hat{Y} = X\beta$$

- Residual

$$e = Y - \hat{Y}$$

- Sum of Squared Error

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = (Y - \hat{Y})^T (Y - \hat{Y}) = (Y - X\beta)^T (Y - X\beta)$$

Least Square Estimation of β

- The estimator β satisfies

$$\frac{\partial}{\partial \beta} ((Y - X\beta)^T (Y - X\beta)) = 0$$

- Assume X is full ranked and $X^T X$ is invertible, then

$$-2X^T(Y - X\beta) = 0$$

$$-X^T Y + (X^T X)\beta = 0$$

$$\beta = (X^T X)^{-1} X^T Y$$

Distribution of β

- Expected value

$$E(\beta) = E\left((X^T X)^{-1} X^T Y\right) = (X^T X)^{-1} X^T E(Y) = (X^T X)^{-1} X^T X \beta = \beta$$

- Variance

$$\begin{aligned} Var(\beta) &= Var\left((X^T X)^{-1} X^T Y\right) \\ &= (X^T X)^{-1} X^T Var(Y) X (X^T X)^{-1} \\ &= (X^T X)^{-1} X^T \sigma^2 I_n X (X^T X)^{-1} \\ &= \sigma^2 (X^T X)^{-1} \end{aligned}$$

Distribution of β

- Assume error term follows identical independent normal distribution (i.i.d.)

$$\epsilon \sim N(0, \sigma^2 I_n)$$

- β is a linear transformation of Y and therefore

$$\beta \sim N(\hat{\beta}, \sigma^2 (X^T X)^{-1})$$

- Unbiased estimate of σ^2 is

$$s^2 = MSE = \frac{SSE}{n - p}$$

- Standard error of β_j will be the j-th diagonal element of $s^2 (X^T X)^{-1}$

Determine if a factor is significant

- Hypothesis Test

$$H_0: \beta_j = 0$$

$$H_1: \beta_j \neq 0$$

- Test statistics:

$$\frac{\beta_j - \hat{\beta}_j}{SE(\beta_j)} \sim t_{n-p}$$

- Interval estimate at a confidence level $1 - \alpha$ (eg. $\alpha = 95\%$)

$$\hat{\beta}_j \pm t_{\alpha/2, n-p} SE(\beta_j)$$

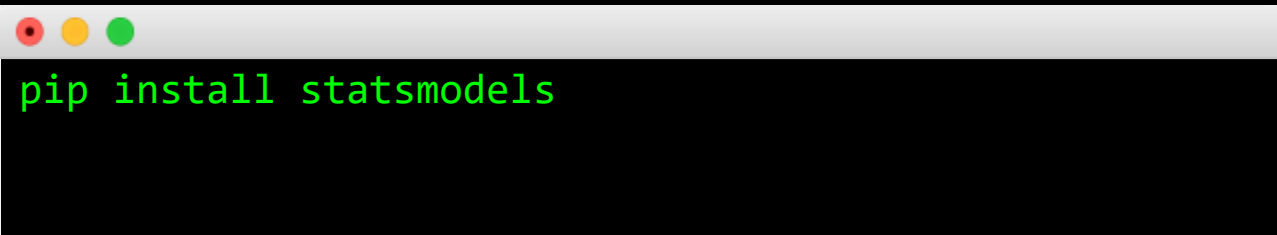
Python Example

- Suppose a stock price is related to economic factors, such as inflation rate, GDP, and interest rate.

Time	Stock Price Y	GDP X1	Interest Rate X2	Inflation Rate X3
1	100	2.5	3.5	1.2
2	105	3	3	1.5
3	110	3.5	2.5	1.3
4	120	4	2	1
5	130	4.5	1.5	0.8
6	140	5	1	0.6
7	150	5.5	0.5	0.4
8	160	6	0	0.2

statsmodels

- Official website (<https://www.statsmodels.org/stable/index.html>)

A terminal window with a light gray title bar containing three colored window control buttons (red, yellow, green). The terminal has a black background and displays the command `pip install statsmodels` in green text.

```
pip install statsmodels
```

Python Example

```
warnings.warn("kurtosistest only valid for n>=20 ... continuing ")
OLS Regression Results

=====
Dep. Variable:      Stock_Price      R-squared:      0.995
Model:              OLS              Adj. R-squared:  0.993
Method:             Least Squares    F-statistic:    493.9
Date:               Fri, 14 Feb 2025  Prob (F-statistic): 1.80e-06
Time:               20:05:00         Log-Likelihood: -14.332
No. Observations:   8               AIC:            34.66
Df Residuals:       5               BIC:            34.90
Df Model:           2
Covariance Type:    nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	6.4238	0.331	19.418	0.000	5.573	7.274
GDP	25.8167	0.249	103.635	0.000	25.176	26.457
Interest_Rate	12.7261	1.858	6.849	0.001	7.950	17.503
Inflation_Rate	-13.1890	4.722	-2.793	0.038	-25.327	-1.051

```
=====
Omnibus:            0.549      Durbin-Watson:      1.556
Prob(Omnibus):      0.760      Jarque-Bera (JB):    0.454
Skew:               0.441      Prob(JB):           0.797
Kurtosis:           2.236      Cond. No.           5.40e+16
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
[2] The smallest eigenvalue is 6.36e-32. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
```

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Sample Data
data = {
    'GDP': [2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0],
    'Interest_Rate': [3.5, 3.0, 2.5, 2.0, 1.5, 1.0, 0.5, 0.0],
    'Inflation_Rate': [1.2, 1.5, 1.3, 1.0, 0.8, 0.6, 0.4, 0.2],
    'Stock_Price': [100, 105, 110, 120, 130, 140, 150, 160]
}
df = pd.DataFrame(data)

# Features (independent variables)
X = df[['GDP', 'Interest_Rate', 'Inflation_Rate']]

# Target variable (dependent variable)
y = df['Stock_Price']

# Add a constant to the model (for the intercept)
X = sm.add_constant(X)

# Creating and fitting the model using statsmodels
model = sm.OLS(y, X).fit()

print(model.summary())
```

Python Example

```
Mean Squared Error: 2.106914370078738
```

```
R-squared: 0.9949638648110163
```

	Actual	Predicted
0	100	99.680118
1	105	102.268701
2	110	111.451772
3	120	121.953740
4	130	131.136811
5	140	140.319882
6	150	149.502953
7	160	158.686024

```
# Making predictions
```

```
y_pred = model.predict(X)
```

```
# Evaluating the model
```

```
mse = mean_squared_error(y, y_pred)
```

```
r2 = r2_score(y, y_pred)
```

```
# Outputting the results
```

```
print("Mean Squared Error:", mse)
```

```
print("R-squared:", r2)
```

```
# Display predicted vs actual values
```

```
results = pd.DataFrame({'Actual': y, 'Predicted': y_pred})
```

```
print(results)
```

Exercise

- Suppose you want to predict the stock price of an electricity company (eg. 0002.HK, 2638.HK)
- What factors can you think of? (i.e. X_1 , X_2 , X_3 , ...)



Exercise

1. Negative factors that could impact operating cost/supply:
 - Coal price – the raw material used to produce electricity
 - Inflation Rate – impact on operational costs and pricing power
 - ...
2. Positive factors that could impact income/demand:
 - Population size
 - Temperature
 - GDP grow rate
 - Gas price – alternative energy
 - ...

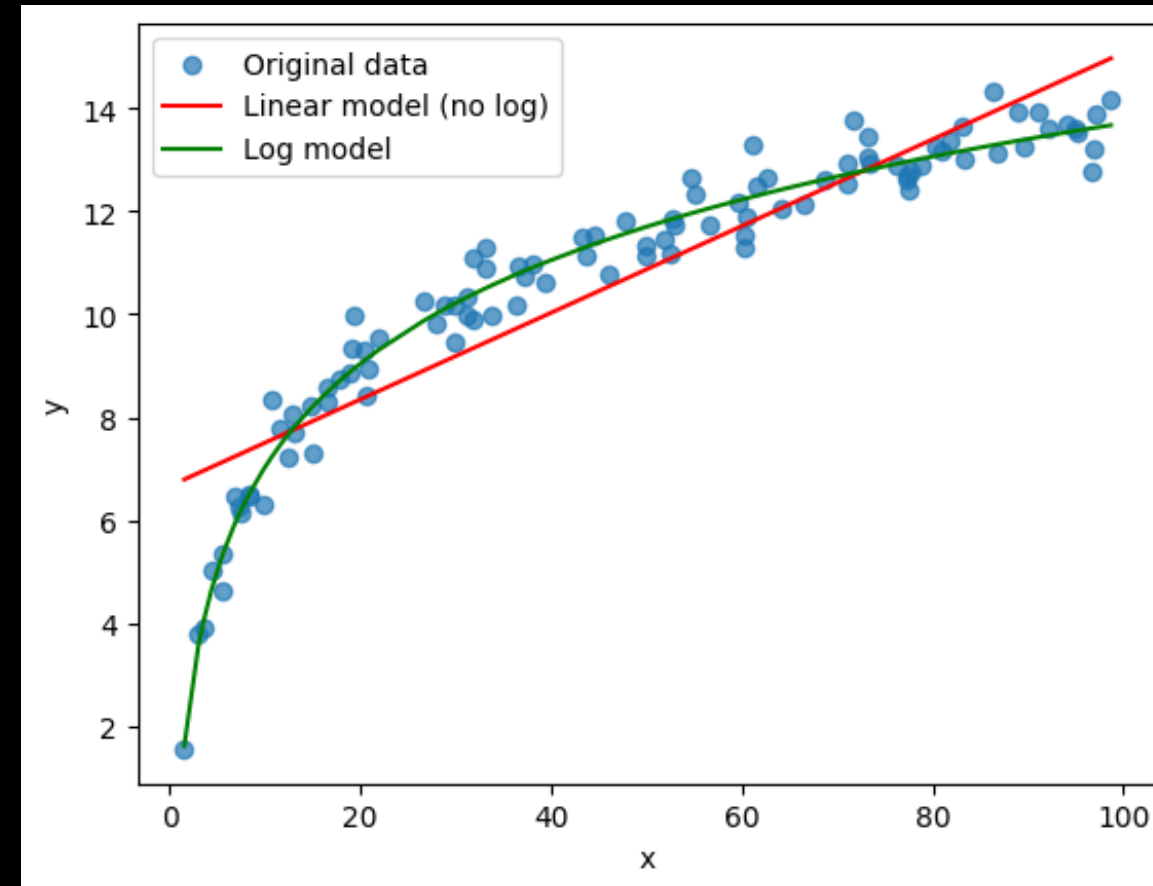
Guidelines for choosing factors

1. Ensure factors have a logical connection to the outcome you want to predict.
2. Use statistical tests (eg. p-values) to assess the significance of each factor.
3. Avoid multicollinearity
 - If some of the independent variables X_i, X_j are highly correlated, X will not be full ranked and thus $X^T X$ is not invertible
4. Data availability
 - Many economic factors such as GDP only release every quarter/year
 - There is time lag in data release
 - May not be suitable for short term trading
5. Simplicity and Interpretability
 - Choose a manageable number of factors to avoid overfitting.
 - Prioritize easily interpretable factors for better insights.
 - Rule of thumb: at least 30 observations for estimating a parameter

Guidelines for choosing factors

6. Data transformation if necessary

- Y and X has no linear relationship. The coefficient may show insignificant in hypothesis test
- However, after log transformation, Y and $\log(X)$ can show a strong linear relationship



ARIMA Model

Introduction to ARIMA Model

ARIMA stands for AutoRegressive Integrated Moving Average

- **AR (AutoRegressive)**: A model that uses the dependency between an observation and a number of lagged observations (previous time steps).
- **I (Integrated)**: Involves differencing the observations to make the time series stationary.
- **MA (Moving Average)**: A model that uses dependency between an observation and a residual error from a moving average model applied to lagged observations.

ARIMA(p,d,q) Model

- ARMA(p, q) is denoted as

$$y_t = \phi_0 + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \theta_1 e_{t-1} + \theta_2 e_{t-2} + \cdots + \theta_q e_{t-q} + e_t$$

$$\text{or} \quad \left(1 - \sum_{i=1}^p \phi_i L^i\right) y_t = \phi_0 + \left(1 - \sum_{i=1}^q \theta_i L^i\right) e_t$$

where

- **p**: number of lag observations (AR part).
- **d**: number of differencing of the original time series (I part).
- **q**: the number of moving average terms (MA part)
- **ϕ** : Coefficients for the autoregressive terms
- **θ** : Coefficients for the moving average terms
- **e_t** : Error term at time t; also called white noise with zero mean and constant variance
- **L** : lag operator
- **$\Delta y_t = y_t - y_{t-1}$**

Model Assumptions

1. The time series is stationary (i.e. mean, variance, and autocorrelation are constant over time).
2. The residuals (errors) are normally distributed.
3. The residuals are uncorrelated (no autocorrelation).

Terminologies

- Sample moment is defined as

$$E(X^r) = \frac{1}{n} \sum_{i=1}^n x_i^r \quad \text{for } r = 1, 2, \dots$$

- Auto-covariance is defined as

$$\gamma_k = \text{Cov}(X_t, X_{t-k}) = \frac{1}{n} \sum_{i=1}^n (X_{t-i} - E(X_t))(X_{t-k-i} - E(X_{t-k})) \quad \text{for lag } k = 1, 2, \dots$$

- Auto-correlation function (ACF) is defined as

$$\rho_k = \frac{\text{Cov}(X_t, X_{t-k})}{\sqrt{\text{Var}(X_t)}\sqrt{\text{Var}(X_{t-k})}} \quad \text{for lag } k = 1, 2, \dots$$

Terminologies

- For a general time series $\{z_t\}$, Partial Auto-correlation function (PACF) is defined as

$$\phi_{1,1} = \text{Corr}(z_{t+1}, z_t) \quad \text{for } k = 1$$

$$\phi_{k,k} = \text{Corr}(z_{t+k} - z'_{t+k}, z_t - z'_t) \quad \text{for } k \geq 2$$

z'_{t+k}, z'_t are linear combination of $\{z_{t+1}, \dots, z_{t+k-1}\}$ that minimize MSE of z_{t+k}, z_t respectively

- By Durbin–Levinson Algorithm,

$$\left\{ \begin{array}{l} \phi_{n,n} = \frac{\rho_n - \sum_{k=1}^{n-1} \phi_{n-1,k} \rho_{n-k}}{1 - \sum_{k=1}^{n-1} \phi_{n-1,k} \rho_k} \\ \phi_{n,k} = \phi_{n-1,k} - \phi_{n,n} \phi_{n-1,n-k} \quad , \text{ for } 1 \leq k \leq n-1 \end{array} \right.$$

Parameter Estimation – AR model

- Consider AR(1): $y_t = \phi_0 + \phi_1 y_{t-1} + e_t$
- There are 2 parameters and need 2 equations to solve

① $E(y_t) = E(\phi_0 + \phi_1 y_{t-1} + e_t) = \phi_0 + \phi_1 E(y_{t-1})$

② $Cov(y_t, y_{t-1}) = Cov(\phi_0 + \phi_1 y_{t-1} + e_t, y_{t-1}) = \phi_1 Var(y_{t-1})$



$$\phi_1 = \widehat{\rho}_1$$

$$\phi_0 = (1 - \widehat{\rho}_1) \mu$$

Parameter Estimation – AR model

For a general AR(p) model,

$$y_t = \phi_0 + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + e_t$$

Yule-Walker Equations

$$\left\{ \begin{array}{l} \rho_1 = \phi_1 \rho_0 + \phi_2 \rho_1 + \cdots + \phi_p \rho_{p-1} \\ \rho_2 = \phi_1 \rho_1 + \phi_2 \rho_0 + \cdots + \phi_p \rho_{p-2} \\ \dots \\ \rho_p = \phi_1 \rho_{p-1} + \phi_2 \rho_{p-2} + \cdots + \phi_p \rho_0 \end{array} \right.$$



Solving the system of linear equations,
we can get $\phi_1, \phi_2, \dots, \phi_p$

Finally, $\phi_0 = (1 - \phi_1 - \phi_2 - \cdots - \phi_p)\mu$

Parameter Estimation – MA model

- Consider MA(1): $y_t = \theta_1 e_{t-1} + e_t$
 - ① $\text{Var}(y_t) = \text{Cov}(y_t, y_t) = \text{Cov}(\theta_1 e_{t-1} + e_t, \theta_1 e_{t-1} + e_t) = (\theta_1^2 + 1) \text{Var}(e_t)$
 - ② $\text{Cov}(y_t, y_{t-1}) = \text{Cov}(\theta_1 e_{t-1} + e_t, y_{t-1}) = \theta_1 \text{Cov}(e_{t-1}, y_{t-1}) = \theta_1 \text{Cov}(e_{t-1}, \theta_1 e_{t-2} + e_{t-1}) = \theta_1 \text{Var}(e_t)$
- We got a quadratic equation in θ_1
$$\rho_1 = \frac{\text{Cov}(y_t, y_{t-1})}{\text{Var}(y_t)} = \frac{\theta_1}{\theta_1^2 + 1}$$
- In general MA(q) model, the parameters for $\{\theta_k\}$ are non linear which could only be solved numerically

Unit Root

- For ARMA(p,q),

$$\left(1 - \sum_{i=1}^p \phi_i L^i\right) y_t = \phi_0 + \left(1 - \sum_{i=1}^q \theta_i L^i\right) e_t$$

- A unit root is present if the polynomial $(1 - \phi_1 x - \phi_2 x^2 - \dots - \phi_p x^p)$ has a root equal to 1
- When unit root exists, the time series is highly persistent over time, meaning that shocks or changes to the series have a long-lasting effect, and autocorrelation decays to zero very slowly.
- A time series is non-stationary if it contains a unit root but the reverse is not true

Unit Root

- Consider the AR(1) process

$$y_t = \phi_0 + \phi_1 y_{t-1} + e_t$$

- It has unit root if $\phi_1 = 1$
- In this case,

$$y_t = \phi_0 t + y_0 + e_1 + e_2 + \cdots + e_t$$

- So the mean will be a function of t , and hence the time series is non-stationary

$$E(y_t) = \phi_0 t + y_0$$

Random Walk Process

- Consider the random walk process which is a special case of AR(1) model

$$y_t = y_{t-1} + e_t$$

- In this case,

$$y_t = y_0 + e_1 + e_2 + \cdots + e_t$$

- The mean is constant but the variance depends on time, and hence non-stationary

$$\left\{ \begin{array}{l} E(y_t) = y_0 \\ Var(y_t) = Var(e_1) * t \end{array} \right.$$

Augmented Dickey Fuller (ADF) Test

- If a price series is mean reverting, then the current price level will tell us something about what the price's next move will be:
 - If the price level is higher than the mean, the next move will be a downward move;
 - if the price level is lower than the mean, the next move will be an upward move.
- The ADF test is based on this observation. We can describe the price changes using a linear model:

$$\Delta y_t = \lambda y_{t-1} + \mu + \beta t + \alpha_1 \Delta y_{t-1} + \cdots + \alpha_k \Delta y_{t-k} + \varepsilon_t$$

Augmented Dickey Fuller (ADF) Test

- When $\lambda=0$, the equation becomes

$$\Delta y_t = \mu + \beta t + \alpha_1 \Delta y_{t-1} + \cdots + \alpha_k \Delta y_{t-k} + \varepsilon_t$$

- This indicates that the change in the series Δy_t is not dependent on its previous value y_{t-1}
- In other words, past values do not predict future changes, which suggests that shocks to the series have permanent effects. This is characteristic of a unit root process (non-stationary).
- The ADF test will find out if there is unit root (i.e. $H_0: \lambda=0$) by looking at the test statistics λ/σ_λ
 - If λ/σ_λ is significantly negative, the time series tends to be mean reverting
 - If λ/σ_λ is significantly positive, the time series tends to be trending

Stationary

- ACF and PACF assume stationarity of the underlying time series. Stationarity can be checked by performing an ADF test:
 - p-value > 0.05: Fail to reject the null hypothesis (H0), the data has a unit root and is non-stationary.
 - p-value <= 0.05: Reject the null hypothesis (H0), the data does not have a unit root and is stationary

```
from statsmodels.tsa.stattools import adfuller
```

```
def check_stationarity(data):  
    result = adfuller(data)  
  
    print('ADF Statistic: %f' % result[0])  
    print('p-value: %f' % result[1])  
    print('Critical Values:')  
    for key, value in result[4].items():  
        print('\t%s: %.3f' % (key, value))  
  
    if (result[1] <= 0.05):  
        print("Stationary")  
    else:  
        print("Non-stationary")
```

```
import random  
data = [random.random() for i in range(0,100)]  
check_stationarity(data)
```

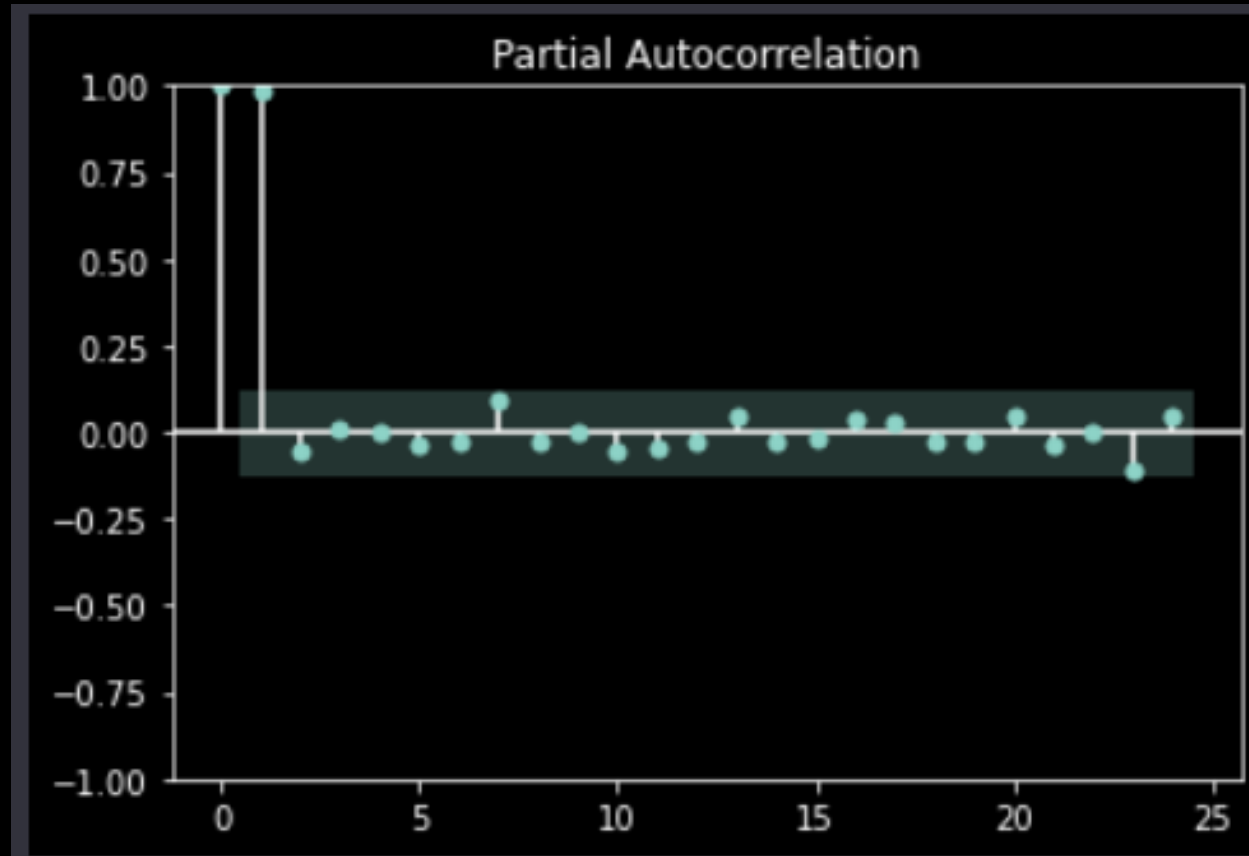
```
ADF Statistic: -8.888879  
p-value: 0.000000  
Critical Values:  
    1%: -3.498  
    5%: -2.891  
   10%: -2.583  
  
Stationary
```

How to determine (p,d,q)?

- The order p and q can be visualized and determined using ACF and PACF charts

Example	PACF
White noise	The partial autocorrelation is 0 for all lags.
Autoregressive model	The partial autocorrelation for an AR(p) model is nonzero for lags less than or equal to p and 0 for lags greater than p.
Moving-average model	• If $\phi_{1,1} > 0$, the partial autocorrelation oscillates to 0. • If $\phi_{1,1} < 0$, the partial autocorrelation geometrically decays to 0.
Autoregressive–moving-average model	An ARMA(p, q) model's partial autocorrelation geometrically decays to 0 but only after lags greater than p.

PACF graph for AR(1)



General rules to determine (p,d,q)

Rules	Identifying the order of differencing and the constant
1	If the series has positive autocorrelations out to a high number of lags (say, 10 or more), then it probably needs a higher order of differencing
2	If the lag-1 autocorrelation is zero or negative, or the autocorrelations are all small and pattern-less, then the series does not need a higher order of differencing. If the lag-1 autocorrelation is -0.5 or more negative, the series may be over-differenced
3	The optimal order of differencing is often the order of differencing at which the standard deviation is lowest. (Not always, though. Slightly too much or slightly too little differencing can also be corrected with AR or MA terms. See rules 6 and 7.)
4	A model with no orders of differencing assumes that the original series is stationary. A model with one order of differencing assumes that the original series has a constant average trend. A model with two orders of total differencing assumes that the original series has a time-varying trend.
5	A model with no orders of differencing normally includes a constant term (which allows for a non-zero mean value). A model with two orders of total differencing normally does not include a constant term. In a model with one order of total differencing, a constant term should be included if the series has a non-zero average trend.

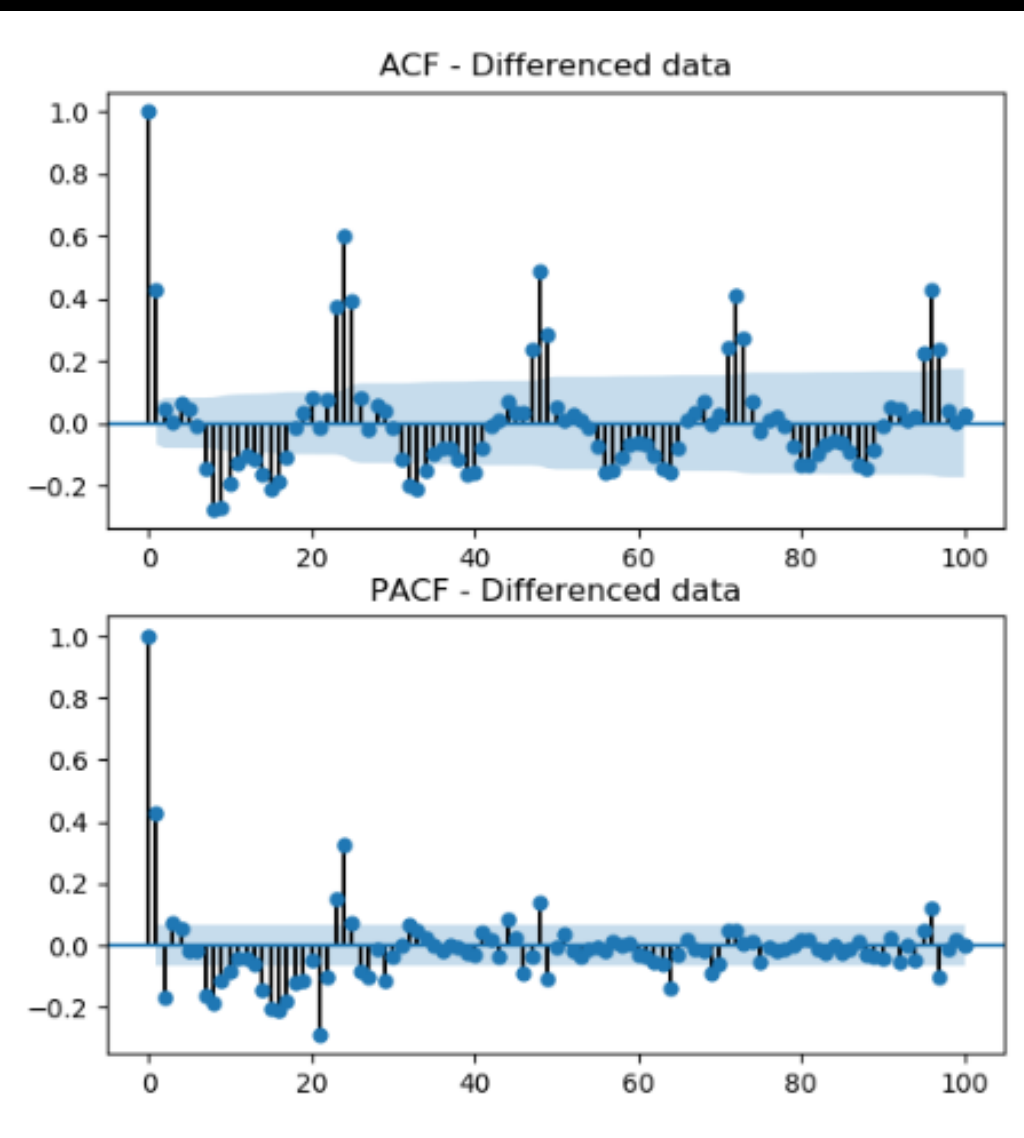
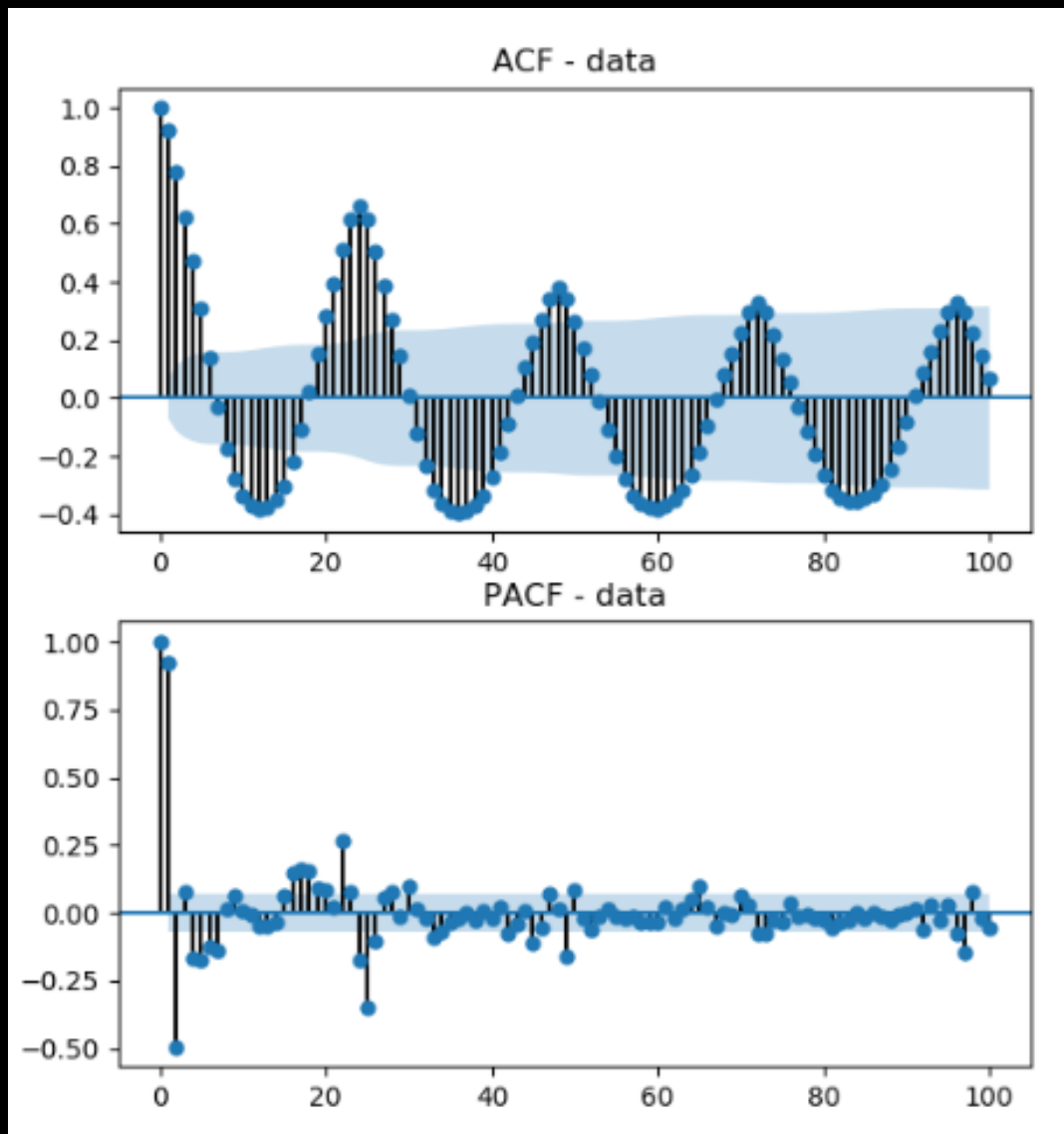
General rules to determine (p,d,q)

Rules	Identifying the numbers of AR and MA terms:
6	If PACF of the differenced series displays a sharp cutoff and/or the lag-1 autocorrelation is positive, then consider adding one or more AR terms to the model. The lag beyond which the PACF cuts off is the indicated number of AR terms.
7	If ACF of the differenced series displays a sharp cutoff and/or the lag-1 autocorrelation is negative, then consider adding an MA term to the model. The lag beyond which the ACF cuts off is the indicated number of MA terms.
8	It is possible for an AR term and an MA term to cancel each other's effects, so if a mixed AR-MA model seems to fit the data, also try a model with one fewer AR term and one fewer MA term (particularly if the parameter estimates in the original model require more than 10 iterations to converge). BEWARE OF USING MULTIPLE AR TERMS AND MULTIPLE MA TERMS IN THE SAME MODEL.
9	If there is a unit root in the AR part of the model (i.e. if the sum of the AR coefficients is almost exactly 1), you should reduce the number of AR terms by one and increase the order of differencing by one.
10	If there is a unit root in the MA part of the model (i.e. if the sum of the MA coefficients is almost exactly 1), you should reduce the number of MA terms by one and reduce the order of differencing by one.
11	If the long-term forecasts appear unstable, there may be a unit root in the AR or MA coefficients

General rules to determine (p,d,q)

Rules	Identifying the seasonal part of the model:
12	If the series has a strong and consistent seasonal pattern, then you must use an order of seasonal differencing (otherwise the model assumes that the seasonal pattern will fade away over time). However, never use more than one order of seasonal differencing or more than 2 orders of total differencing (seasonal + nonseasonal).
13	If the autocorrelation of the appropriately differenced series is positive at lag s , where s is the number of periods in a season, then consider adding an SAR term to the model. If the autocorrelation of the differenced series is negative at lag s , consider adding an SMA term to the model. The latter situation is likely to occur if a seasonal difference has been used, which should be done if the data has a stable and logical seasonal pattern. The former is likely to occur if a seasonal difference has not been used, which would only be appropriate if the seasonal pattern is not stable over time. You should try to avoid using more than one or two seasonal parameters (SAR+SMA) in the same model, as this is likely to lead to overfitting of the data and/or problems in estimation.

What are the (p,d,q) in this example?



What are the (p,d,q) in this example?

Possible choices

- Non-seasonal:
 - (1, 1, 0) or (2, 0, 0)
 - (0, 1, 1) or (0, 0, 2)
 - (1, 1, 1)
- Seasonal difference at lag 24
 - (1, 1, 0)
 - (0, 1, 1)
 - (1, 1, 1)

Python Implementation

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

```
plt.style.use('dark_background')
```

```
# Fetch historical data for Microsoft (MSFT)
data = yf.download('MSFT', start='2023-01-01', end='2023-12-31')
data = data[['Close']]
```

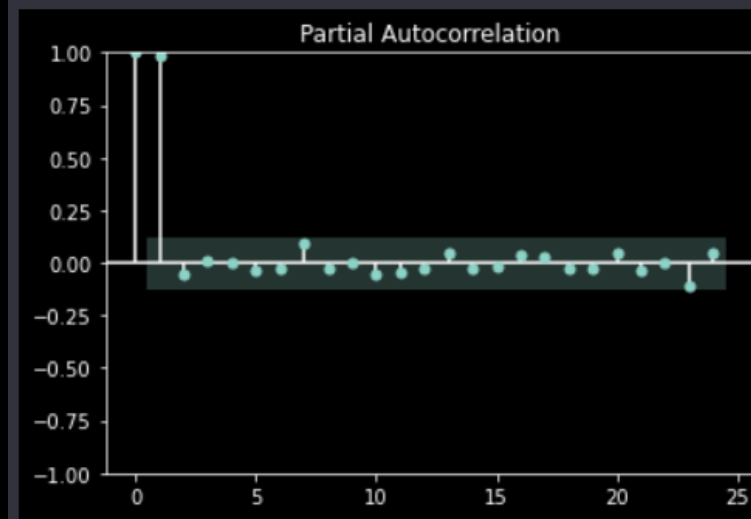
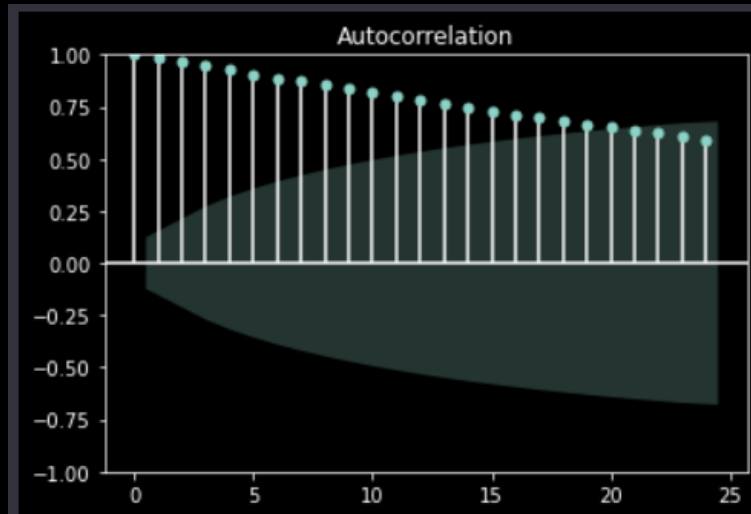
```
# Visualize the data
plt.figure(figsize=(10, 6))
plt.plot(data)
plt.title('Time Series Data')
```

```
plt.show()
```

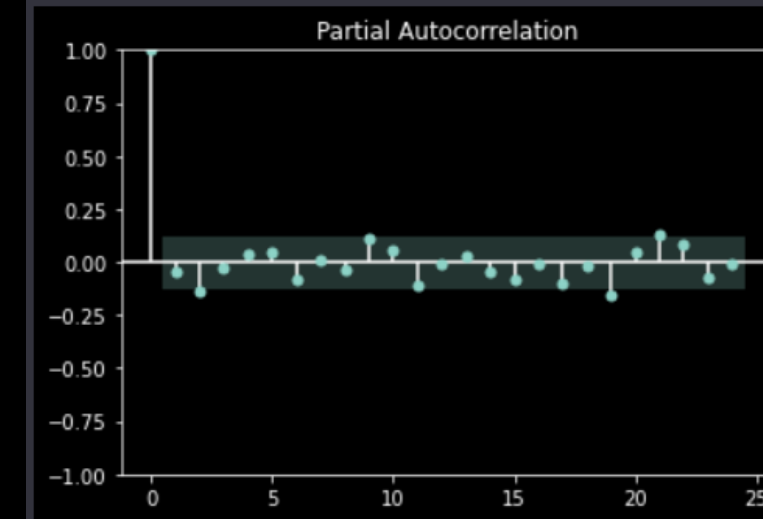
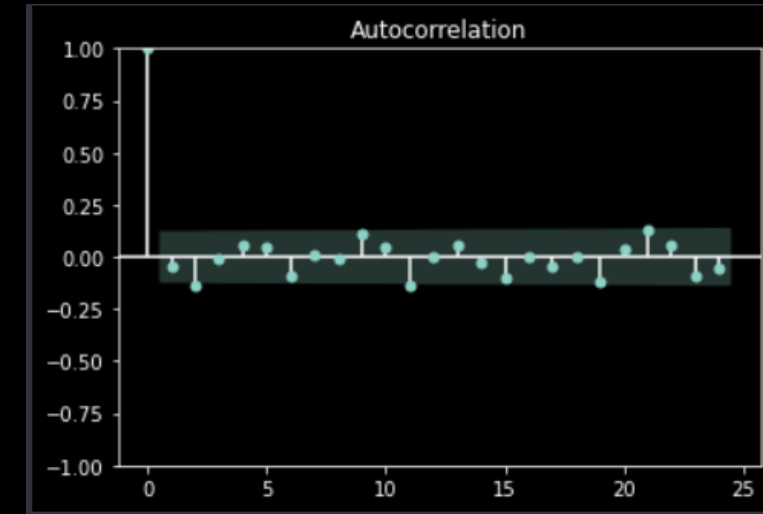
```
# Plot ACF and PACF to determine p and q
plot_acf(data)
plot_pacf(data)
plt.show()
```

```
# differencing
data_diff = data.diff().dropna()
plot_acf(data_diff)
plot_pacf(data_diff)
plt.show()
```

d=0



d=1



Python Implementation

```
p = 1
d = 0
q = 1
```

```
# Fit ARIMA model
```

```
model = ARIMA(data, order=(p, d, q))
model_fit = model.fit()
```

```
# Summary of the model
```

```
print(model_fit.summary())
```

```
# Forecasting
```

```
forecast = model_fit.forecast(steps=10)
print(forecast)
```

```
SARIMAX Results
=====
Dep. Variable:      Close_MSFT      No. Observations:      250
Model:              ARIMA(1, 0, 1)   Log Likelihood         -743.796
Date:              Thu, 05 Feb 2026   AIC                    1495.592
Time:              22:46:24          BIC                    1509.678
Sample:            0                 HQIC                   1501.261
                  - 250
Covariance Type:    opg
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
const         308.0829      50.418         6.111      0.000       209.265       406.900
ar.L1          0.9970       0.007       143.205      0.000         0.983         1.011
ma.L1         -0.0427       0.056        -0.761      0.446        -0.153         0.067
sigma2         22.0274       1.710       12.885      0.000        18.677        25.378
=====
Ljung-Box (L1) (Q):              0.00      Jarque-Bera (JB):              7.39
Prob(Q):                        0.99      Prob(JB):              0.02
Heteroskedasticity (H):          0.71      Skew:              0.19
Prob(H) (two-sided):            0.13      Kurtosis:             3.75
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
250      370.289148
251      370.105472
252      369.922339
253      369.739747
254      369.557694
255      369.376179
256      369.195199
257      369.014754
258      368.834842
259      368.655460
Name: predicted_mean, dtype: float64
```

ARIMA Trading Strategy

- Construct a stationary time series and fit an ARIMA model
- Strategy 1:
 - Take ARIMA model's predicted values as the fair/reference market price.
 - If current market price is much lower than the fair price, open LONG position
 - If current market price is much higher than the fair price, open SHORT position
- Strategy 2:
 - For AR(p) model, project for the future prices
 - If the future price is projected to be higher than the current price, open LONG position
 - If the future price is projected to be lower than the current price, open SHORT position

Backtest ARIMA Strategy

- Instrument: MSFT
- Initial capital: 10k USD
- Period: 2023.01.01 – 2023.12.31
- Data Interval: 1 day
- Short-selling: No

```
from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA

class AlgoEvent:
    def __init__(self):
        self.p, self.d, self.q = 1, 0, 1

    def start(self, mEvt):
        self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)
        self.evt.start()

    def on_marketdatafeed(self, md, ab):
        # get previous 100 days closing price
        obs = self.evt.getHistoricalBar(contract={"instrument": "MSFT"}, numOfBar=100, interval="D", timestamp=md.timestamp)
        arr_close = [obs[t]["c"] for t in obs]

        # put data into dataframe
        data = pd.DataFrame(arr_close)

        # fit ARIMA model
        model = ARIMA(data, order=(self.p, self.d, self.q))
        model_fit = model.fit()

        # 5 day forecast
        forecast = model_fit.forecast(steps=5)
        future_price = forecast.iloc[-1]
        self.evt.consoleLog(forecast)

        # open buy order if future price is higher than current price
        if future_price > md.askPrice:
            order = AlgoAPIUtil.OrderObject(
                instrument = md.instrument,
                openclose = 'open',
                buysell = 1, #buy
                ordertype = 0, #market order
                volume = 1, #1 lot
                holdtime = 24*60*60*5 #holding period for 5 days
            )
            self.evt.sendOrder(order)
```

Backtest ARIMA Strategy

Monthly Return:

Year	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	YTD
2023	0.16%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.09%	0.05%	0.81%	0.24%	0.0%	1.35%

Performance Statistics: ⓘ

No. of Tradable Days:	272	No. of Win Days:	30	No. of Loss Days:	32
Win Rate:	48.3871%	Max. Consecutive Win Day:	8	Max. Consecutive Loss Day:	6
Odd Ratio:	0.9375	Max. Consecutive Gains:	55.48	Max. Consecutive Loss:	-42.67
No. of Trades:	82	Average Consecutive Win Day:	0.54	Average Consecutive Loss Day:	0.25
Total PnL:	133.94	Average Per Trade PnL:	1.63	Average Per Day PnL:	0.49
Mean Daily Return:	0.0049%	Median Daily Return:	0.0%	Mean Annual Return:	1.2426%
Daily Return StdDev:	0.0649%	25th percentile Daily Return:	0.0%	75th percentile Daily Return:	0.0%
Daily Return Downside StdDev:	0.0335%	95% 1 day return VaR:	-0.0423%	99% 1 day return VaR:	-0.1868%
Daily Sharpe Ratio:	0.076	Annual Sharpe Ratio:	1.2065	Max. Drawdown Amount:	42.67
Daily Sortino Ratio:	0.1471	Annual Sortino Ratio:	2.3352	Max. Drawdown Percent:	0.4267%
Max. Drawdown Duration:	160	Average Drawdown Duration:	47.95	Annual Volatility:	1.0258%
Gross Profit:	213.495	Gross Loss:	-79.554	Profit Factor:	2.6836
Jensen Alpha:	0.0	Beta:	0.0	Information Ratio:	0.0
Omega Ratio:	0.0	Treynor Ratio:	0.0	Tail Ratio:	1.7807
Calmar Ratio:	2.9123	Average Holding Day:	5.2439	Annual Turnover Rate:	110.6951%

Hurst Exponent

Hurst Exponent

- The speed of diffusion can be characterized by the variance

$$Var(\tau) = \langle |z(t + \tau) - z(t)|^2 \rangle$$

where

- $z = \log(y)$ is the log price
- τ is an arbitrary time lag
- $\langle |...| \rangle$ is an average over all t

- For a geometric random walk, we know the variance is proportional to τ

$$\langle |z(t + \tau) - z(t)|^2 \rangle \sim \tau$$

- This equation won't hold for mean reverting or trending series.

Hurst Exponent

- We can re-write it as

$$\langle |z(t + \tau) - z(t)|^2 \rangle \sim \tau^{2H}$$

- This is the definition of Hurst Exponent. The series exhibit as
 - geometric random walk If $H = 0.5$
 - mean reverting if $H < 0.5$
 - trending if $H > 0.5$
- H can be an indicator for the degree of mean reverting or trending
 - If H is close to 0, it will be more mean reverting
 - If H is close to 1, it will be more trending

Variance Ratio Test

- Hypothesis test:
 - $H_0: H = 0.5$ (i.e. random walk)
 - $H_1: H \neq 0.5$
- It simply tests whether below equal to 1.

$$\frac{\text{Var}(z(t) - z(t - \tau))}{\tau \text{Var}(z(t) - z(t - 1))} \sim \tau^{2H-1}$$

```
import numpy as np
from arch.unitroot import VarianceRatio

# Generate some sample data
np.random.seed(42)
data = np.cumsum(np.random.randn(200)) + 100

# Perform the Variance Ratio test
vr_test = VarianceRatio(y=data, lags=2)

# Print the test results
print(vr_test.summary())
```

Mean Reversion Process

- In practice, it is important to know how quick a time series revert to its mean
- Let's consider the Ornstein Uhlenbeck formula for a mean reverting process

$$dy_t = \kappa(\theta - y_t) dt + \sigma d\varepsilon$$

- $\kappa > 0$ is the rate of mean reversion
- θ is the long terms mean
- $\sigma > 0$ is the volatility of the Brownian process

- Solution: for any $0 < s < t$,

$$y_t = \theta + (y_s - \theta)e^{-\kappa(t-s)} + \sigma \int_s^t e^{-\kappa(t-u)} dW_u$$

$$E(y_t) = \theta + (y_s - \theta)e^{-\kappa(t-s)}$$

$$\text{Var}(y_t) = \frac{\sigma^2}{2\kappa} (1 - e^{-2\kappa(t-s)})$$

Half Life of Mean Reversion

- Note that the expected value decay exponentially to θ

$$E(y_t) = \theta + (y_s - \theta)e^{-\kappa(t-s)}$$

- Consider

$$\frac{1}{2}(y_0 - \theta) = (y_0 - \theta)e^{-\kappa t}$$

- Half life will be

$$t = \frac{\log(2)}{\kappa}$$

Parameter Estimation

- Consider the Ornstein Uhlenbeck in a discrete form

$$dy_t = \kappa(\theta - y_t) dt + \sigma d\varepsilon$$

$$\Delta y_t = \kappa\theta - \kappa y_{t-1} + \varepsilon_t$$

- We can fit a linear regression model Δy_t against y_{t-1} where $-\kappa$ is the slope and $\kappa\theta$ is the constant term

How to discover Mean Reversion?

- We can base on Hurst Exponent and Variance ratio test to determine whether a time series is mean reverting
- The connection of the coefficient κ and half life is also very useful
 - If κ is estimated to be negative, that means the time series is not mean reverting at all and we shouldn't attempt to apply a mean reversion strategy to trade
 - If κ is very close to zero, that means the half life is very long and a mean reverting strategy will not be very profitable as we can't complete many round trades in a given period
 - If κ is a natural time scale. For example, if the half life is 20 days, we shouldn't use a look-back of 5 days to compute a moving average or standard deviation for a mean-reversion strategy. Often, setting the lookback to equal a small multiple of the half-life is close to optimal, and doing so will allow us to avoid brute-force optimization of a free parameter based on the performance of a trading strategy

How to discover Momentum?

- Momentum means that past returns are positively correlated with future returns. Thus we can just calculate the correlation coefficient of the returns together with its p-value
- Sometimes the most positive correlations are between returns of different lags. For example, 1-day returns might show negative correlations, while the correlation between past 20-day return with the future 40-day return might be very positive. We should find the optimal pair of past and future periods that gives the highest positive correlation and use that as our look-back and holding period for our momentum strategy.
- We can test for the correlations between the signs of past and future returns. This is appropriate when all we want to know is that an up move will be followed by another up move, and we don't care whether the magnitudes of the moves are similar
- If we are interested instead in finding out whether there is long-term trending behavior without regard to specific time frames, we can calculate the Hurst exponent together with the Variance Ratio test to rule out the null hypothesis of a random walk

Backtest Mean Reversion Strategy

- Fit a linear regression model according to the discrete form of Ornstein Uhlenbeck formula
- Check model fitness based on R^2 (for example, assume the model is good if $R^2 > 0.5$)
- Compute the theoretical half-life $t_{0.5} = \log(2)/\kappa$
- We can assume it is a proper mean reversion period if $t_{0.5}$ is between 1 to 10
- Take the regression's predicted values as the fair/reference market price
 - If current market price is much lower than the fair price, open LONG position
 - If current market price is much higher than the fair price, open SHORT position
- Also set holding period to $t_{0.5}$

Backtest Mean Reversion Strategy

- Instrument: MSFT
- Initial capital: 10k USD
- Period: 2023.01.01 – 2023.12.31
- Data Interval: 1 day
- Short-selling : No

```

from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from math import log

```

```

class AlgoEvent:
    def __init__(self):
        pass

    def start(self, mEvt):
        self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)
        self.evt.start()

```

```

def on_marketdatafeed(self, md, ab):
    # get previous 100 days closing price
    obs = self.evt.getHistoricalBar(contract={"instrument": "MSFT"}, numOfBar=10, interval="D", timestamp=md.timestamp)
    close = np.array([obs[t]['c'] for t in obs])

    #  $Y(t) - Y(t-1) = a + b * Y(t-1)$ 
    Y = close[1:] - close[:-1]
    X = close[:-1]

    # Reshape for sklearn which expects a 2D array for features
    X = X.reshape(-1, 1)

    # fit ARIMA model
    model = LinearRegression()
    model.fit(X, Y)

    # evaluate model effectiveness
    Y_pred = model.predict(X)
    mse = mean_squared_error(Y, Y_pred)
    r2 = r2_score(Y, Y_pred)
    self.evt.consoleLog('mse, r-square = ', mse, r2)

    # skip entry if e-square too small
    if r2 < 0.5: return

    # compute half life
    b0 = model.intercept_
    b1 = model.coef_[0]
    t_halflife = -log(2)/b1
    self.evt.consoleLog('half-life = ', t_halflife)

    # skip entry if half life is negative or too large
    if t_halflife <= 1 or t_halflife > 10: return

    # open buy order if current ask price is lower than model price
    if md.askPrice < Y_pred[-1] + close[-1]:
        order = AlgoAPIUtil.OrderObject(
            instrument = md.instrument,
            openclose = 'open',
            buysell = 1, #buy
            ordertype = 0, #market order
            volume = 1, #0.01 lot
            holdtime = 24*60*60*t_halflife #unit in second
        )
        self.evt.sendOrder(order)

```

Backtest Mean Reversion Strategy

Monthly Return:

Year	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	YTD
2023	0.0%	-0.02%	0.04%	0.0%	0.0%	0.04%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.06%

Performance Statistics: ⓘ

No. of Tradable Days:	272	No. of Win Days:	3	No. of Loss Days:	6
Win Rate:	33.3333%	Max. Consecutive Win Day:	2	Max. Consecutive Loss Day:	4
Odd Ratio:	0.5	Max. Consecutive Gains:	5.83	Max. Consecutive Loss:	-4.04
No. of Trades:	6	Average Consecutive Win Day:	1.33	Average Consecutive Loss Day:	0.06
Total PnL:	5.85	Average Per Trade PnL:	0.98	Average Per Day PnL:	0.02
Mean Daily Return:	0.0003%	Median Daily Return:	0.0%	Mean Annual Return:	0.065%
Daily Return StdDev:	0.005%	25th percentile Daily Return:	0.0%	75th percentile Daily Return:	0.0%
Daily Return Downside StdDev:	0.0019%	95% 1 day return VaR:	0.0%	99% 1 day return VaR:	-0.0076%
Daily Sharpe Ratio:	0.0516	Annual Sharpe Ratio:	0.819	Max. Drawdown Amount:	4.04
Daily Sortino Ratio:	0.1377	Annual Sortino Ratio:	2.1853	Max. Drawdown Percent:	0.0404%
Max. Drawdown Duration:	4	Average Drawdown Duration:	0.06	Annual Volatility:	0.0791%
Gross Profit:	9.878	Gross Loss:	-4.023	Profit Factor:	2.4554
Jensen Alpha:	0.0	Beta:	0.0	Information Ratio:	0.0
Omega Ratio:	0.0	Treynor Ratio:	0.0	Tail Ratio:	0.0
Calmar Ratio:	1.608	Average Holding Day:	2.6667	Annual Turnover Rate:	7.7609%

Martingale Strategy

For random walk market

Introduction to Martingale Strategy

- It is a betting system that originated in 18th century France and was popularized in the 19th century.
- The system was named after a casino owner in London named John Henry Martindale, who encouraged players to use the strategy to win at his casino.



Casino Example

- **Betting process:**
 - The player places a \$1 bet on red. If the ball lands on black, he lose the bet.
 - The player doubles their bet to \$2 and places it on red again. If the ball lands on black again, he lose \$2.
 - The player doubles their bet to \$4 and places it on red again. If the ball lands on black again, he lose \$4.
 - The player doubles their bet to \$8 and places it on red again. If the ball lands on red this time, he win \$8 and have recouped the previous losses and made a \$1 profit.

Bet	Outcome	Total Profit/Loss
\$1	Loss	-\$1
\$2	Loss	-\$3
\$4	Loss	-\$7
\$8	Win	+\$1

The mathematics

- Assume initial bet size is \$1, the betting sequence will be

$$\{1, 2, 4, 8, 16, \dots, 2^n\}$$

- Suppose the player loses for n consecutive rounds and win at $n+1$ round,
 - Sum of all previous losses = $1 + 2 + 4 + \dots + 2^{(n-1)} = 2^n - 1$
 - Win at $(n+1)$ th round = 2^n
 - So the total profit/loss will be +1
- The probability of losing n consecutive rounds is 0.5^n . The probability of losing 10 consecutive rounds is only 0.000976 which means that this strategy is highly likely to succeed in short term

Strategy Assumptions & Limitations

- Only 2 possible outcomes (i.e. win or lose)
- The probability of win and lose are equal
- Each round's outcome is independent
- It requires unlimited capital in long run

Martingale Strategy Example

- Instrument: XAUUSD
- Backtest period: 2023.01 – 2023.12
- Initial Capital: US\$50k
- Open 0.01 lot buy order every time (for XAUUSD, 1 lot = 100 shares)
- Set TP and SL to be \$10

```
from AlgoAPI import AlgoAPIUtil, AlgoAPI_Backtest

class AlgoEvent:
    def __init__(self):
        self.bet_amt = 10
        self.bet_size_base = 0.01
        self.bet_size = self.bet_size_base
        self.entry_price = None

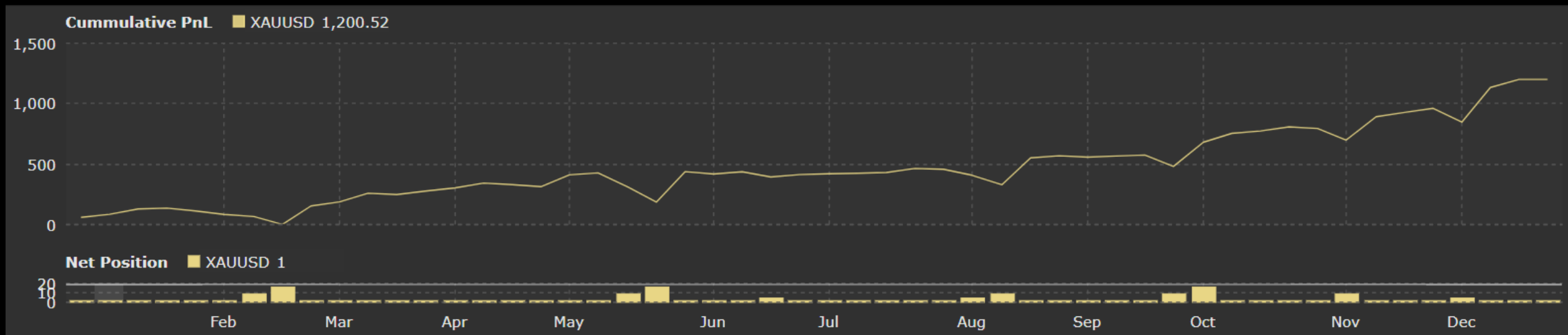
    def start(self, mEvt):
        self.evt = AlgoAPI_Backtest.AlgoEvtHandler(self, mEvt)
        self.evt.start()

    def on_marketdatafeed(self, md, ab):
        # open new trade if no position
        pos, _, _ = self.evt.getSystemOrders()
        if pos[md.instrument]["netVolume"]==0:
            order = AlgoAPIUtil.OrderObject(
                instrument = md.instrument,
                openclose = 'open',
                buysell = 1, # buy
                ordertype = 0, # market order
                volume = self.bet_size,
                takeProfitLevel = md.askPrice + self.bet_amt,
                stopLossLevel = md.askPrice - self.bet_amt
            )
            self.evt.sendOrder(order)

    def on_orderfeed(self, of):
        if of.status=="success":
            if of.openclose=="open":
                self.entry_price = of.fill_price
            elif of.openclose=="close":
                # reset to base size if win
                if of.fill_price > self.entry_price:
                    self.bet_size = self.bet_size_base
                # double bet size if lose
            else:
                self.bet_size*=2
```

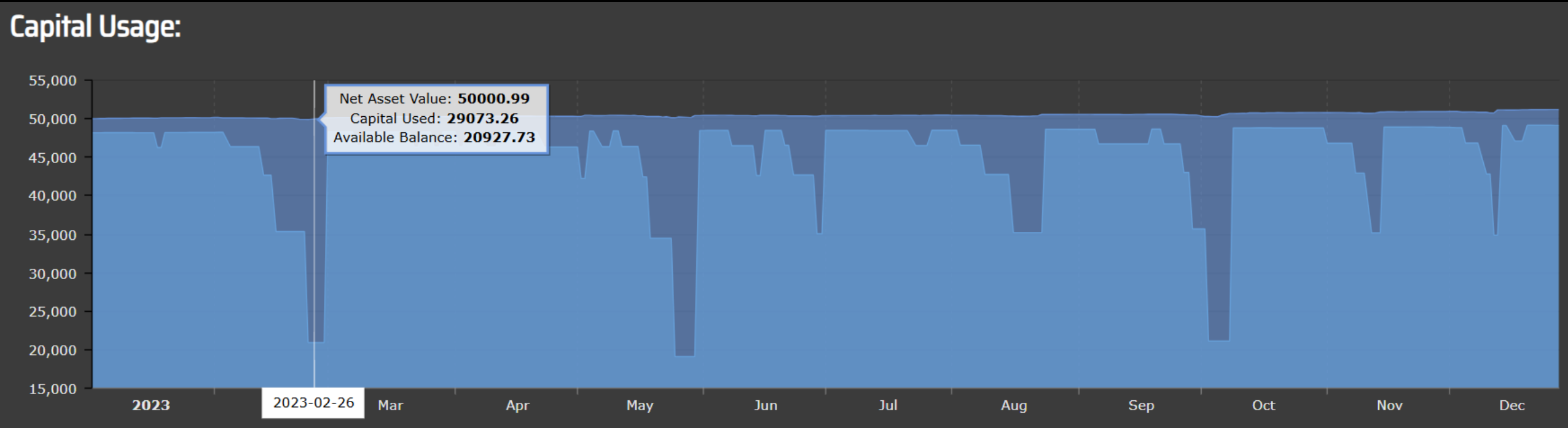
Martingale Strategy Example

- This strategy is effective in making stable profit



Martingale Strategy Example

- But need to carefully manage the capital in case of consecutive losses



Ways to enhance Martingale Strategy

- Entry only when the current market is detected to be random
- Rather than always opening a buy order,
 - Enter long position if we detect a short term up trend
 - Enter short position if we detect a short term down trend
- TL/SL size adjustable according to market liquidity

Key Takeaways

- Market is generally classified into
 - Trending (Momentum)
 - Mean Reversion
 - Random Walk
- We discussed the following statistical models/ trading strategies
 - Simple Linear Regression
 - Multiple Linear Regression
 - ARIMA
 - Hurst Exponent (a good indicator for market classification)
 - Martingale (random walk)